A project report on

KEY-CONDITIONED SECURE EMERGENT COMMUNICATION FOR MULTI-ROBOT SYSTEMS AGAINST LEARNING-BASED EAVESDROPPERS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science Engineering with specialization in Artificial Intelligence and Robotics

by

ADITYA VASU RANA (23BDS1020)

HARSH VIKRAM SINGH (23BRS1216)

HARSHIT SHARMA (23BRS1109)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November,2026

KEY-CONDITIONED SECURE EMERGENT COMMUNICATION FOR MULTI-ROBOT SYSTEMS AGAINST LEARNINGBASED EAVESDROPPERS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science Engineering with specialization in Artificial Intelligence and Robotics

by

ADITYA VASU RANA (23BDS1020)

HARSH VIKRAM SINGH (23BRS1216)

HARSHIT SHARMA (23BRS1109)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "KEY-CONDITIONED SECURE EMERGENT COMMUNICATION FOR MULTI-ROBOT SYSTEMS AGAINST LEARNING-BASED EAVESDROPPERS" submitted by Harsh Vikram Singh (23BRS1216) for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Asha Jerlin M.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: Signature of the Candidate

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "**Key-Conditioned Secure Emergent Communication for Multi-Robot Systems Against Learning-Based Eavesdroppers"** is prepared and submitted by Harsh Vikram Singh (23BRS1216) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science Engineering (Artificial Intelligence & Robotics)** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Asha Jerlin M

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,
Computer Science Engineering (Artificial Intelligence & Robotics)

Name: Dr. D. Sathian

Date

ABSTRACT

Agents that learn a discrete code can coordinate with very short messages. The weakness is stability: if a symbol always stands for the same task state, anyone listening on the channel can log enough labelled traffic to read the code without ever touching the sender's observations. This report tests episode-dependent symbol remapping as a lightweight confidentiality measure for learned robot communication.

Alice, the sender, quantizes her latent vector against a 16-entry codebook. Before the chosen index leaves her, it passes through a permutation of the codebook indices, and that permutation is rebuilt for every episode from a shared key and the episode index hashed with SHA-256. Bob holds the same key, rebuilds the same permutation and inverts it. Eve sees every transmitted index. She has neither the key nor the per-episode permutation.

Four configurations were run in a simulated 7 × 7 Grid World with ten task states: plain transmission, a fixed permutation (V1), a key-dependent static permutation (V2 Static) and the key-dependent rotating permutation (V2 Rotating). Bob's task-state accuracy shows whether the protocol still works. An analytical eavesdropper and a neural one, an MLP with two hidden layers over one-hot symbols, show what leaks. Under plain, V1 and V2 Static, both eavesdroppers reached 100.00%. Under V2 Rotating they fell to 9.32% (analytical) and 9.36% (neural), against a 10% reference for uniform guessing over ten balanced states, while Bob decoded at 100%. All of these figures are preliminary. Standalone attacker scripts and the combined benchmark disagree in places, and that disagreement has not yet been reconciled. Within the two tested attackers, rotating the mapping per episode closed the symbol-to-state shortcut that the static schemes left open. It gives no guarantee against an attacker who tracks episode boundaries, reads message sequences or knows the keygeneration procedure.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Asha Jerlin M, Associate Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for her constant guidance, continual encouragement, understanding; more than all, she taught me patience in my endeavor. My association with her is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of intrusion detection and network security.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to **Dr. D. Sathian**, Head of the Department, B.Tech. Computer Science Engineering (Artificial Intelligence & Robotics) and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staff at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **Harsh Vikram Singh**

CONTENTS

ABSTRACT ..... **i**

ACKNOWLEDGEMENT ..... **ii**

CONTENTS ..... **iii**

LIST OF FIGURES ..... **vi**

LIST OF TABLES ..... **vii**

LIST OF ACRONYMS ..... **viii**

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND…… 1
1.2 PROBLEM STATEMENT…… 1
1.3 OBJECTIVES…… 2
1.4 OVERALL SYSTEM ARCHITECTURE…….. 2
1.5 COMMUNICATION CONFIGURATIONS…….. 3
1.6 SCOPE……
3
1.7 ORGANISATION OF THE REPORT…… 3

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION…….. 4
2.2 LEARNED MULTI-AGENT COMMUNICATION……. 4
2.3 EMERGENT COMMUNICATION AND DISCRETE REPRESENTATIONS……5
2.4 VECTOR QUANTIZATION IN LEARNED COMMUNICATION……. 5
2.5 PRIVACY IN MULTI-AGENT COMMUNICATION…….. 6
2.6 LEARNING-BASED COMMUNICATION SECURITY…….. 6
2.7 COMPARISON OF RELATED RESEARCH……..... 7
2.8 RESEARCH GAP……... 8
2.9 CHAPTER SUMMARY……..... 8

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 INTRODUCTION……. 9
3.2 EXISTING SYSTEM……..... 9
3.3 PROPOSED SYSTEM……. 10
3.4 SYSTEM REQUIREMENTS……... 11
3.5SYSTEM ARCHITECTURE……. 12
3.6 COMMUNICATION CONFIGURATIONS…… 13
3.7 THREAT MODEL…… 13
3.7.2 ASSUMPTIONS AND EXCLUSIONS……. 13
3.8 SYSTEM WORKFLOW……... 14
3.9 EVALUATION CRITERIA…….. 15
3.10 CHAPTER SUMMARY……. 15

CHAPTER 4

PROPOSED METHODOLOGY

4.1 INTRODUCTION……. 16
4.2 METHODOLOGY OVERVIEW…….. 17
4.3 LEARNED DISCRETE COMMUNICATION MODEL……. 17
4.4 EPISODE-DEPENDENT REMAPPING…….. 18
4.5 COMMUNICATION CONFIGURATIONS…… 19
4.6 EAVESDROPPER DESIGN…… 19
4.7 EVALUATION METRICS……... 21
4.8 ALGORITHM 4.1: V2 ROTATING……. 22
4.9 METHODOLOGY SUMMARY…….. 22

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION……. 23
5.2 DEVELOPMENT ENVIRONMENT……... 23
5.3 DATASET AND SIMULATION…….. 24

5.4 SENDER MODEL…… 24
5.5 REMAPPING MODULE……..
26 5.6 RECEIVER MODEL……
29
5.7 EAVESDROPPER IMPLEMENTATION……... 29
5.8 EXPERIMENTAL WORKFLOW…… 31
5.9 IMPLEMENTATION CHALLENGES…… 31
5.10 CHAPTER SUMMARY……. 32

CHAPTER 6

RESULTS AND DISCUSSION

6.1 INTRODUCTION……. 33
6.2 EXPERIMENTAL CONFIGURATION…….. 33
6.3 BOB ACCURACY…… 34
6.4 EVE ACCURACY…… 35
6.5 STATIC VERSUS ROTATING REMAPPING…… 37
6.6 RANDOM-GUESSING REFERENCE…… 37
6.7 LIMITATIONS AND VALIDITY CHECKS……... 37
6.8 CHAPTER SUMMARY……... 38

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION……. 39
7.2 ACHIEVEMENTS…… 39
7.3 LIMITATIONS……. 40
7.4 FUTURE WORK…….. 40
7.5 FINAL REMARKS……... 41
REFERENCES…….. 42

APPENDIX A: DATASET GENERATION AND PROJECT CONFIGURATION... 44

APPENDIX B: ALICE–BOB COMMUNICATION ARCHITECTURE…… 47

APPENDIX C: EPISODE-DEPENDENT SYMBOL REMAPPING……... 50

APPENDIX D: TRAINING, EVALUATION AND EAVESDROPPER ANALYSIS…… 53

APPENDIX E: EXPERIMENTAL FRAMEWORK, SECURITY ANALYSIS AND LIMITATIONS…… 58

LIST OF FIGURES

1.1 OVERALL SYSTEM ARCHITECTURE OF SECURE ROBOT COMMUNICATION 2

3.1 ARCHITECTURE DIAGRAM OF BOB AND ALICE 11

3.2 ALICE–BOB–EVE COMMUNICATION MODEL 14

4.1 ARCHITECTURE OF THE LEARNED DISCRETE COMMUNICATION MODEL 16

5.1 SYMBOL-STATE MAPPING AFTER VECTOR QUANTIZATION OF EMERGENT COMMUNICATION 25

5.2 ANALYTICAL EVE'S ATTACK ON PLAIN COMMUNICATION 26

5.3.1 FIXED MAPPING (V1) COMMUNICATION RESULTS 27

5.3.2 ANALYTICAL EVE ATTACK ON FIXED MAPPING (V1) COMMUNICATION 27

5.4 V2 MODE COMMUNICATION RESULTS 28

5.5 COMPARISON OF V2 STATIC VS V2 ROTATING COMMUNICATION MODE 29

5.6.1 ANALYTICAL EVE ATTACK ON V2-STATIC COMMUNICATION MODE 30

5.6.2 ANALYTICAL EVE ATTACK ON V2-ROTATING COMMUNICATION MODE 31

6.1 ACCURACY (BOB'S DECODING ACCURACY) FOR COMMUNICATION MODES 34

6.2 COMPARISON OF EVE ATTACKS ON DIFFERENT COMMUNICATION MODES 35

LIST OF TABLES

1.1 COMMUNICATION CONFIGURATIONS USED IN THE EVALUATION 3

2.1 SUMMARY OF RELATED RESEARCH 7

3.1 MAIN SYSTEM COMPONENTS AND THEIR FUNCTIONS 12

3.2 COMPARISON OF COMMUNICATION CONFIGURATIONS 13

4.1 COMMUNICATION CONFIGURATIONS USED IN THE PROPOSED METHODOLOGY 20

5.1 SOFTWARE AND DEVELOPMENT COMPONENTS 23

5.2 MAIN SIMULATION CHARACTERISTICS 24

5.3 NEURAL EAVESDROPPER ARCHITECTURE 30

6.1 EXPERIMENTAL CONFIGURATION 33

6.2 RECEIVER ACCURACY ACROSS COMMUNICATION CONFIGURATIONS 34

6.3 PRELIMINARY EAVESDROPPER ACCURACY 35

6.4 QUALITATIVE COMPARISON OF REMAPPING BEHAVIOUR 37

LIST OF ACRONYMS

AI	Artificial Intelligence
ANN	Artificial Neural Network
DL	Deep Learning
MLP	Multilayer Perceptron
PRNG	Pseudorandom Number Generator

SHA-256	Secure Hash Algorithm 256-bit
V1	Version 1
V2	Version 2
VQ	Vector Quantization
COE	Centre of Excellence
CPU	Central Processing Unit
GPU	Graphics Processing Unit
ML	Machine Learning
NN	Neural Network
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve

Chapter 1

Introduction

1.1 BACKGROUND

A robot in a team rarely sees the whole environment. What one agent observes, another often needs for its next decision, so multi-agent robotic systems exchange messages to split the work and converge on a joint objective without any single robot holding the full state. Hand-written protocols fix the message format in advance. Learned communication replaces that format with a neural encoder that compresses each observation into whatever the downstream task needs, and the receiver is trained jointly to read it. When messages must come from a small vocabulary instead of a full observation vector, the code becomes discrete: the sender picks one symbol out of a finite set, which keeps bandwidth low and makes every message trivially easy to log.

Discreteness has a side effect. If one symbol always stands for one task state, the code is a lookup table, and a listener who logs intercepted symbols next to the states they preceded can rebuild that table without ever seeing the sender's observation. Nothing in the learning process prevents this.

This project changes the table every episode. Alice still quantizes her observation into one of 16 codebook entries, but the index she transmits is first passed through a permutation that a shared key and the episode index determine. Bob, holding the key, inverts that permutation and hands the original index to his decoder. Eve observes only the permuted index. Without the key, what index 3 meant in one episode tells her nothing direct about what it means in the next.

1.2 PROBLEM STATEMENT

A fixed link between transmitted symbols and task states lets an eavesdropper learn what the messages mean. Applying one fixed permutation to the whole vocabulary does not change that: it relabels the symbols, and after enough episodes Eve learns the relabeled table as easily as the original one. The question here is narrower and testable. If the permutation is redrawn for every episode, does Eve's task-state inference collapse while Bob still decodes correctly?

Four configurations answer it. Plain transmission sends the quantized index unchanged, V1 applies one fixed permutation, V2 Static applies a permutation derived once from the shared key, and V2 Rotating derives a fresh permutation from the key and the episode index. Two attackers, one analytical and one neural, try to recover the task state from whatever crosses the channel. Bob's accuracy is measured alongside theirs, because a scheme that hides the state from Eve by also hiding it from Bob has solved nothing.

1.3 OBJECTIVES

The project set out to:

1. Build a learned discrete communication pipeline in which a vector quantizer turns Alice's latent vector into one of 16 codebook indices.
2. Permute that index before transmission, with the permutation fixed by a shared key and the episode index.
3. Let Bob rebuild the same permutation from the key, invert it and recover the original index.
4. Train an analytical eavesdropper and a neural eavesdropper to infer the task state from intercepted indices.
5. Compare unprotected transmission with the fixed (V1) and keyed (V2 Static, V2 Rotating) permutations.
6. Report what the mechanism achieves, and where it stops, under the stated simulation and threat model.

1.4 OVERALL SYSTEM ARCHITECTURE

Three parties take part. Alice encodes an observation into a latent vector and snaps it to the nearest entry of a learned codebook, which yields a discrete index. Before transmission, the V2 Rotating module permutes that index according to the shared key and the current episode number.

On the far side of the channel, Bob regenerates the permutation, inverts it and passes the recovered index to his decoder. Eve taps the same channel. She sees exactly the index that Bob receives, but without the key she cannot tell which permutation produced it.

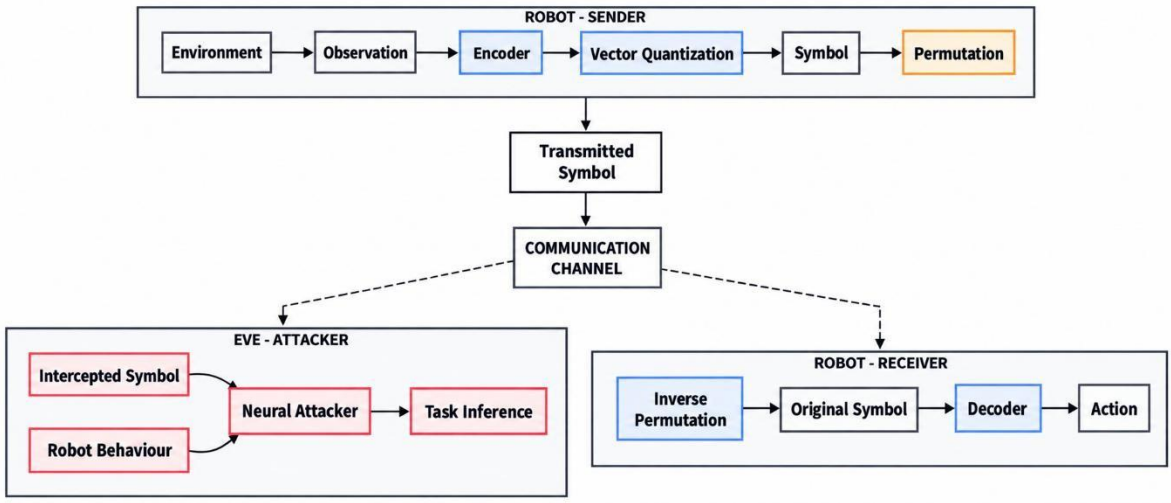


Figure 1.1 — Overall System Architecture of Secure Robot Communication

1.5 COMMUNICATION CONFIGURATIONS

The project evaluates four communication configurations to examine how the stability of symbol mappings affects an eavesdropper's ability to infer task states.

Table 1.1 Communication configurations used in the evaluation

Configuration	**Description**
Plain	Original discrete symbols are transmitted without remapping.
Fixed	A single permutation is applied and remains unchanged across episodes.
Key-dependent Static	A permutation derived from a shared key remains constant across episodes.
V2 Rotating (Proposed)	A permutation is derived using the shared key and episode index, changing the symbol mapping across episodes.

Setting V1 and V2 Static beside V2 Rotating separates two effects that are easy to conflate: applying a permutation at all, and changing it between episodes. Each configuration is scored on Bob's task-state accuracy and on the accuracy of the analytical and neural eavesdroppers.

1.6 SCOPE OF THE PROJECT

Everything in this project runs in the form of simulation. Observations are 16-dimensional, the task has ten states and the codebook has total 16 different codebook entries. Inside scope: the learned pipeline, deterministic permutation generation, inverse mapping at Bob, and task-state inference by the analytical and neural eavesdroppers, all compared across the four configurations under one threat model. Outside it: any claim of cryptographic security. Compromised keys, attacks on the endpoints, active tampering with messages, replay, and attackers who bring extra context such as episode boundaries or message histories were not tested, and the results say nothing about them.

1.7 ORGANISATION OF THE REPORT

Seven chapters run from motivation to measurement. Chapter 1 sets out the problem, objectives, scope and architecture. Related work on learned communication, vector quantization, privacy and eavesdropping is reviewed in Chapter 2, and Chapter 3 fixes the requirements, the design and the threat model that every later experiment assumes. Chapter 4 gives the remapping method and its algorithm. Chapter 5 covers implementation. Chapter 6 reports results and their limits, and Chapter 7 closes with conclusions and future work

Chapter 2

Literature Review

2.1 INTRODUCTION

Communication between multiple intelligent agents has become an important research area in multi-agent systems, robotics, and reinforcement learning. When agents operate in a shared environment, the ability to exchange relevant information can improve coordination and support effective decision-making. Traditional communication approaches often rely on predefined message formats, whereas learned communication allows agents to develop representations suited to their tasks through neural network training.

An important direction in this field is emergent communication, in which agents learn to communicate using signals or symbols without being explicitly provided with a complete communication language. Such systems can employ continuous message vectors or discrete vocabularies. Discrete communication is particularly relevant when messages must be represented compactly or when an agent needs to select from a finite set of possible symbols.

Although learned communication offers flexibility, it also introduces challenges related to the interpretation and privacy of transmitted messages. If an external observer can repeatedly associate a transmitted symbol with a particular task state, the observer may learn the communication pattern and infer the information being exchanged. This motivates the investigation of techniques that change the relationship between transmitted symbols and their meanings while preserving communication between the intended agents.

This chapter reviews research related to learned multi-agent communication, discrete representations, vector quantization, privacy-preserving communication, and neural approaches to communication security. It concludes by identifying the research gap addressed by the proposed episode-dependent symbol remapping mechanism.

2.2 LEARNED MULTI-AGENT COMMUNICATION

Learned multi-agent communication enables agents to develop communication strategies using neural networks and learning algorithms. Instead of relying entirely on manually specified messages, agents learn representations that support coordination in a particular environment.

Foerster et al. investigated learning communication protocols in deep multi-agent reinforcement learning through approaches that allow agents to improve their communication during training. Their work demonstrated the importance of learning message representations jointly with agent behaviour.

Sukhbaatar et al. proposed CommNet, a neural architecture that enables multiple agents to exchange information through differentiable communication. Information from other agents is incorporated into the computation of each agent, allowing communication and decisionmaking to be trained together.

Other approaches have explored more selective communication. TarMAC introduced targeted multi-agent communication, allowing agents to learn which information to send and

which agents should receive it. Attention-based communication mechanisms similarly help agents identify information that is useful for cooperative tasks.

These studies establish the importance of learned communication in cooperative systems. However, their primary emphasis is on communication and coordination rather than the specific problem of protecting the meanings of discrete transmitted symbols from an external eavesdropper.

2.3 EMERGENT COMMUNICATION AND DISCRETE REPRESENTATIONS

Emergent communication refers to the development of communication conventions through interactions between agents. In these systems, agents learn signals that help them coordinate without necessarily being given a predefined human-readable language.

Research by Lazaridou et al. investigated the emergence of communication through cooperative multi-agent games. Havrylov and Titov studied how agents can learn to communicate through sequences of discrete symbols. These studies illustrate how meaningful communication conventions can develop through learning.

Discrete communication represents messages using a finite vocabulary. Compared with unrestricted continuous representations, discrete symbols provide an explicit set of possible messages that can be transmitted between agents. Nevertheless, the meaning of a symbol is determined by the learned protocol and the context in which the symbol is used.

A stable communication convention can create a potential weakness. When a particular symbol repeatedly represents the same task state, an observer may learn the association between the symbol and the state. The problem becomes especially relevant when the transmitted symbols can be collected and labelled during training or evaluation.

The present project investigates this issue in a learned robot communication setting. Instead of attempting to eliminate the learned communication protocol, it changes the identifiers of the selected symbols before transmission. The intended receiver reverses this transformation so that the original learned protocol can continue to operate.

2.4 VECTOR QUANTIZATION IN LEARNED COMMUNICATION

Vector quantization is a technique for representing a continuous vector using one entry from a finite collection of codebook vectors. It is used in machine learning to obtain discrete latent representations from continuous neural network outputs.

Van den Oord et al. introduced the Vector-Quantized Variational Autoencoder, which uses a learned codebook to represent continuous latent information using discrete codes. This approach provides a foundation for systems that need to transform neural representations into finite symbolic representations.

Carmeli et al. investigated quantized communication in emergent communication settings, examining how discrete representations can be used in learned protocols. Liu et al. also studied adaptive discrete communication bottlenecks using dynamic vector quantization, addressing differences in the representational requirements of communicating agents. These approaches are relevant to the present project because vector quantization provides a mechanism for converting the sender's latent representation into a discrete symbol. In the

implemented system, Alice produces a latent representation from a 16-dimensional observation, and the vector quantizer selects a symbol from a codebook containing 16 entries.

The resulting symbol is the input to the proposed remapping mechanism. It is important to distinguish the two operations: vector quantization selects the original symbol, whereas episode-dependent remapping changes the identifier transmitted to the receiver. The remapping operation does not, by itself, retrain or change the underlying learned vocabulary.

2.5 PRIVACY IN MULTI-AGENT COMMUNICATION

Privacy is an important consideration in multi-agent systems because exchanged messages may contain information about observations, internal states, or task objectives. An agent may need to share information with a cooperating agent while limiting what an external observer can infer.

Research on privacy-preserving multi-agent reinforcement learning has investigated ways to reduce the exposure of information during communication. Zhang et al. studied privacy preservation in peer-to-peer communication for multi-agent reinforcement learning. Other surveys have reviewed privacy-preserving mechanisms for multi-agent systems and the challenges associated with protecting information in distributed environments.

The problem considered in this project is narrower than general privacy preservation. It focuses on whether an eavesdropper can infer a task-state label from an intercepted discrete symbol. The attacker does not receive the sender's original observation and is not provided with the current episode-specific permutation.

This setting enables a controlled comparison of symbol mappings. Plain communication provides a baseline in which the original symbol is transmitted directly. Fixed and static permutations change the transmitted identifiers but maintain a stable mapping. The episodevarying approach changes the mapping across episodes, potentially making associations learned from earlier episodes less useful.

2.6 LEARNING-BASED COMMUNICATION SECURITY

Machine learning has also been investigated for communication security. Abadi and Andersen proposed adversarial neural cryptography, in which neural networks learn to protect messages against an adversarial observer. Meraouche et al. explored learning asymmetric encryption using adversarial neural networks.

These approaches provide relevant background for studying the relationship between learned representations and communication security. However, neural encryption and symbol remapping are not equivalent. A cryptographic encryption scheme is designed around specific security requirements, whereas the mechanism proposed in this project changes identifiers within a learned discrete vocabulary.

Classical cryptography, including the foundational work of Diffie and Hellman on publickey cryptography, provides established approaches to secure communication. The present project does not attempt to replace such mechanisms. Instead, it evaluates whether

episodedependent remapping affects the task-state inference accuracy of specified eavesdroppers in a simulated environment.

The distinction is important when interpreting experimental results. An eavesdropper accuracy close to a random-guessing reference indicates limited classification performance under the tested conditions, but it does not establish that the communication is cryptographically secure or that no information can be inferred.

2.7 COMPARISON OF RELATED WORK

The following table summarises selected research directions and their relevance to the project.

Table 2.1 Summary of related research

Research direction	**Representative work**	**Relevance to this project**
Learned multi-agent communication	Foerster et al.; Sukhbaatar et al.	Provides background on agents learning communication protocols.
Targeted communication	TarMAC; attentionbased communication	Demonstrates selective message exchange between agents.
Emergent communication	Lazaridou et al.; Havrylov and Titov	Explores the development of learned communication conventions.
Vector quantization	Van den Oord et al.	Provides the basis for converting latent representations into discrete symbols.
Quantized communication	Carmeli et al.; Liu et al.	Examines discrete representations in learned communication systems.
Privacy-preserving multi-agent systems	Zhang et al.; privacy surveys	Motivates limiting information exposure during communication.
Neural communication security	Abadi and Andersen; Meraouche et al.	Provides background on learningbased approaches to communication protection.
Classical cryptography	Diffie and Hellman	Establishes the distinction between symbol remapping and cryptographic security.

2.8 RESEARCH GAP

Existing research has investigated learned multi-agent communication, emergent symbolic protocols, vector quantization, privacy-preserving mechanisms, and learning-based security. These areas provide the foundation for studying the relationship between communication representations and the information exposed to an observer.

However, the specific question addressed by this project is how the stability of a learned discrete symbol mapping affects an eavesdropper's ability to infer task states. A fixed permutation changes the identifiers associated with the learned symbols, but the mapping remains constant across episodes. Consequently, an observer who collects enough labelled samples may learn the transformed associations.

The project investigates episode-dependent remapping as an alternative. A shared key and episode index determine the permutation used during a communication episode. Alice applies the permutation before transmission, and Bob uses the inverse permutation to recover the original symbol. The analytical and neural eavesdroppers attempt to infer task states from transmitted identifiers without being given the current permutation.

The principal research gap is therefore the empirical evaluation of whether changing symbol mappings across episodes reduces task-state inference while maintaining the intended receiver's prediction accuracy under a defined threat model.

2.9 CHAPTER SUMMARY

This chapter reviewed research on learned multi-agent communication, emergent communication, vector quantization, privacy preservation, and learning-based communication security. The reviewed literature establishes the relevance of learned discrete representations and highlights the potential exposure of message semantics when symbol mappings remain stable.

Based on this background, the project investigates episode-dependent symbol remapping using a shared key and episode index. The next chapter describes the system requirements, design, communication configurations, and threat model used to evaluate the proposed approach.

Chapter 3

System Analysis And Design

3.1 INTRODUCTION

System analysis and design define the structure, requirements, and operational behaviour of the proposed secure robot communication system. The purpose of this project is to examine whether changing the identifiers of learned discrete communication symbols across episodes

can reduce task-state inference by an external eavesdropper while preserving the intended receiver's ability to interpret messages.

The system is designed around three entities: Alice, who generates and transmits discrete symbols; Bob, who receives and decodes the transmitted symbols; and Eve, who observes the communication channel and attempts to infer the underlying task state. A vector quantizer converts Alice's latent representation into a discrete symbol, while the proposed V2 Rotating mechanism applies an episode-dependent permutation before transmission.

The design separates the learned communication process from the remapping operation. This separation allows the same underlying communication protocol to be evaluated under multiple configurations without treating the remapping mechanism as a replacement for the learned encoder, codebook, or decoder.

3.2 EXISTING SYSTEM

In the baseline communication system, Alice converts an observation into a latent representation and uses vector quantization to select a discrete symbol. The selected symbol is transmitted directly to Bob, who uses the corresponding learned representation to predict the task state.

The principal limitation of this arrangement is that the transmitted identifiers remain consistent across communication episodes. If a symbol repeatedly corresponds to a particular task state, an eavesdropper may collect intercepted samples and learn the relationship between the symbols and their associated states.

A fixed permutation changes the transmitted identifiers but does not remove this consistency. Once an attacker learns the transformed symbol-to-state relationship, the attacker may continue to infer task states in subsequent episodes.

3.2.1 LIMITATIONS OF THE EXISTING SYSTEM

The major limitations considered in this project are:

1. Stable symbol associations: Repeated use of the same mapping may allow an eavesdropper to learn the meaning of transmitted symbols.
2. Limited protection from fixed remapping: A single permutation changes the identifiers but preserves a consistent mapping over time.
3. Dependence on attacker observations: The success of an eavesdropper depends on the samples available for learning the symbol-to-state association.
4. Potential information exposure: The intended receiver and an external observer may both observe the transmitted symbol, although only the receiver has access to the information needed to reverse the episode-specific mapping.
5. Need for empirical evaluation: A remapping mechanism must be evaluated against different attackers to determine whether it reduces task-state inference under the specified conditions.

3.3 PROPOSED SYSTEM

The proposed system introduces episode-dependent symbol remapping into the learned discrete communication pipeline. Alice first generates a discrete symbol using the trained encoder and vector quantizer. A permutation derived from the shared key and current episode index is then applied to the symbol before transmission.

Let s denote the original discrete symbol and π_e denote the permutation associated with episode e. The transmitted symbol c is defined as

$$\mathbf{c = \pi_e(s)}$$

Bob derives the same episode-specific permutation and applies its inverse:

$$\mathbf{s = \pi_e^{-1}(c)}$$

After recovering the original symbol, Bob continues with the existing codebook lookup and decoding process. Eve observes c but is not given the shared key or current permutation. The analytical and neural attackers are evaluated according to their ability to predict the task-state label from the intercepted symbols.

The proposed method is compared with plain communication, fixed permutation, and keydependent static permutation. This comparison helps determine whether the observed change in attacker performance is associated with the use of remapping generally or with changing the mapping between episodes.

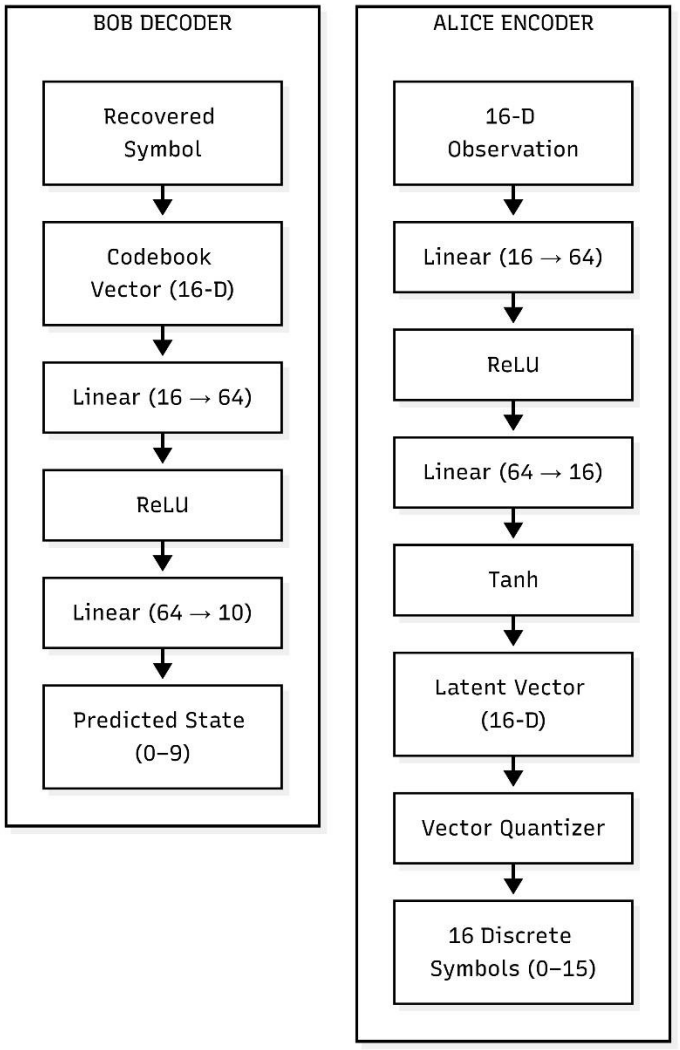


Figure 3.1 Architecture diagram of Bob and Alice

3.4 SYSTEM REQUIREMENTS

3.4.1 FUNCTIONAL REQUIREMENTS

The system must provide the following functions:

- Accept a 16-dimensional task observation as input to the sender.
- Transform the observation into a latent representation using a neural encoder.
- Select a discrete symbol using a vector quantizer with a 16-entry codebook.
- Generate the appropriate symbol mapping for the selected communication configuration.
- Apply the permutation to the original symbol before transmission.
- Allow Bob to recover the original symbol using the inverse permutation.
- Predict the task state using the receiver decoder.

- Allow Eve to observe transmitted symbol identifiers and attempt task-state inference.
- Calculate and compare receiver and eavesdropper accuracy across configurations.

3.4.2 NON-FUNCTIONAL REQUIREMENTS

The system should satisfy the following non-functional requirements:

- Reliability: Bob should recover the correct original symbol when the matching permutation is applied.
- Reproducibility: The same shared key and episode index should produce the same permutation under the implemented deterministic generation procedure.
- Modularity: The remapping mechanism should remain separate from the learned encoder and decoder.
- Evaluability: The system should support consistent measurement of receiver and eavesdropper accuracy.
- Computational feasibility: The experiments should be executable in a software simulation without requiring physical robotic hardware.
- Transparency: The implementation should clearly define the attacker's information access and the limitations of the security evaluation.

3.5 SYSTEM ARCHITECTURE

The architecture consists of the sender pipeline, remapping module, communication channel, receiver pipeline, and eavesdropper.

Table 3.1 Main system components and their functions

Component	**Function**
Alice encoder	Converts the input observation into a latent representation.
Vector quantizer	Selects a discrete symbol from the learned codebook.
Remapping module	Applies the permutation associated with the current configuration and episode.
Communication channel	Carries the transmitted symbol identifier to Bob and exposes it to Eve under the threat model.
Inverse mapping module	Allows Bob to recover the original discrete symbol.
Bob decoder	Predicts the task state from the recovered symbol representation.
Analytical eavesdropper	Estimates symbol-to-state associations from intercepted samples.
Neural eavesdropper	Uses a trained classifier to predict task states from transmitted symbols.

The modular structure supports comparison between the four communication configurations. The encoder, codebook, and receiver form the learned communication model, while the mapping applied before transmission determines the identifiers observed by Eve.

3.6 COMMUNICATION CONFIGURATIONS

Four configurations are included in the system design.

Table 3.2 Comparison of communication configurations

Configuration	**Mapping behaviour**	**Purpose**
Plain	Original symbol is transmitted directly.	Establishes the baseline inference performance.
Fixed permutation (V1)	A fixed permutation is used across episodes.	Evaluates the effect of changing identifiers without changing the mapping over time.
Key-dependent static permutation (V2 Static)	A permutation derived from the shared key remains constant across episodes.	Evaluates static key-dependent remapping.
Key-dependent rotating permutation (V2 Rotating)	The permutation depends on the shared key and episode index.	Evaluates whether episodedependent changes affect eavesdropper inference.

The configurations are evaluated using receiver accuracy and eavesdropper accuracy. Comparisons are meaningful only when the same evaluation protocol, data split, and taskstate definitions are used consistently.

3.7 THREAT MODEL

The threat model defines the information available to Eve and the attacks considered in the evaluation.

3.7.1 EAVESDROPPER CAPABILITIES

Eve is assumed to be capable of:

- Observing the transmitted symbol identifiers.
- Collecting intercepted samples for training and evaluation.
- Learning symbol-to-state associations from observed samples.
- Training a neural classifier to predict task-state labels.
- Evaluating its predictions on held-out intercepted messages.

For the V2 Rotating configuration, Eve is not provided with the shared key or current episode-specific permutation.

3.7.2 THREAT MODEL ASSUMPTIONS

The evaluation assumes that Alice and Bob are not compromised and that the shared key remains unavailable to Eve. It does not consider endpoint compromise, active message modification, replay attacks, denial-of-service attacks, or attackers with access to the secret key.

The current evaluation also does not establish resistance to all context-aware or sequenceaware attackers. In particular, the ability of Eve to infer episode boundaries or exploit additional information may affect attack performance.

Consequently, the threat model supports an empirical evaluation of task-state inference from intercepted symbols rather than a formal guarantee of confidentiality.

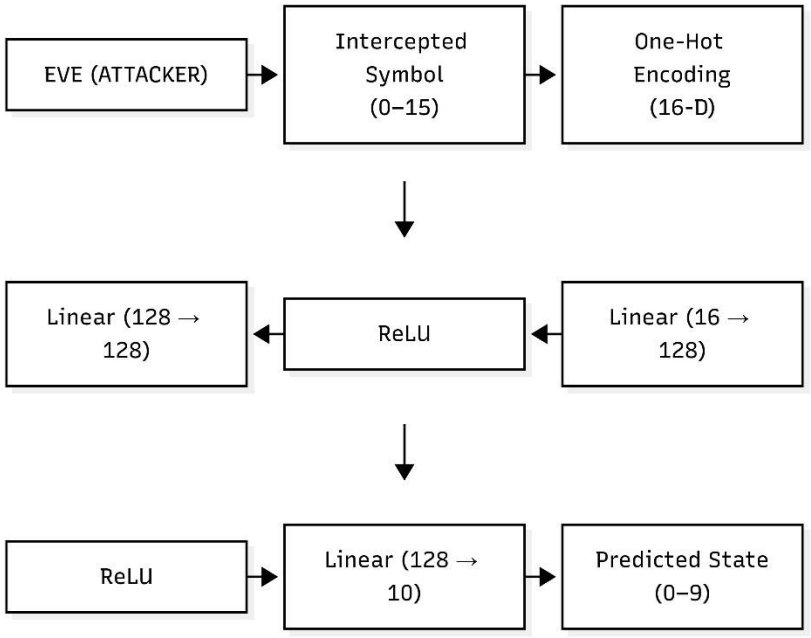


Figure 3.2 Architecture of Neural Eve Attacker

3.8 SYSTEM WORKFLOW

The system operates through the following sequence:

1. Alice receives the task observation.
2. The encoder transforms the observation into a latent representation.
3. The vector quantizer selects a discrete symbol from the codebook.
4. The remapping module determines the permutation for the selected configuration.

5. Alice applies the permutation and transmits the resulting symbol identifier.
6. Bob derives the corresponding permutation and applies the inverse mapping.
7. Bob uses the recovered symbol to predict the task state.
8. Eve observes the transmitted identifier and attempts to infer the task state.
9. Receiver and eavesdropper predictions are evaluated using the selected accuracy metrics.

The same general workflow is used to compare the communication configurations, with the remapping behaviour changing according to the selected baseline.

3.9 EVALUATION CRITERIA

The principal evaluation criteria are receiver accuracy and eavesdropper accuracy.

Receiver accuracy measures the proportion of evaluated examples for which Bob predicts the correct task state. Eavesdropper accuracy measures the proportion for which Eve correctly predicts the task-state label.

For a balanced classification task with ten states, uniform random guessing provides a reference accuracy of 10%. This reference is useful for interpreting the results, although it does not establish that an attacker has learned no information.

The evaluation should also consider the consistency of results across repeated runs, the distribution of task-state labels, and the availability of episode information to the attacker. Because discrepancies have been observed between experimental scripts, the numerical findings should remain preliminary until the evaluation protocol has been reconciled.

3.10 CHAPTER SUMMARY

This chapter presented the system analysis and design for learned robot communication with episode-dependent symbol remapping. It described the existing communication approach, limitations of stable symbol mappings, functional and non-functional requirements, system components, communication configurations, threat model, and evaluation criteria. The proposed design separates the learned communication protocol from the remapping operation. Alice generates a discrete symbol, V2 Rotating changes its transmitted identifier according to the episode-specific permutation, and Bob reverses the transformation before decoding. Eve attempts to infer task states from intercepted symbols without access to the current mapping.

The next chapter describes the proposed methodology in detail, including the mathematical formulation of vector quantization, permutation generation, inverse mapping, and the evaluation of analytical and neural eavesdroppers.

Chapter 4

Proposed Methodology

4.1 INTRODUCTION

The proposed methodology investigates whether episode-dependent symbol remapping can reduce an eavesdropper's ability to infer task states from learned discrete communication. The approach combines a neural communication model, vector quantization, deterministic permutation generation, inverse mapping at the receiver, and attacker evaluation.

The underlying communication protocol converts an input observation into a discrete symbol using a learned encoder and codebook. Instead of transmitting this symbol directly, the proposed V2 Rotating mechanism changes its transmitted identifier according to a permutation associated with the current episode. Alice and Bob derive the same permutation using a shared key and episode index. Alice applies the permutation before transmission, and Bob applies the inverse permutation after reception.

Eve observes the transmitted symbol identifier but is not provided with the current permutation. Two attacker approaches, analytical and neural, are used to evaluate the extent to which task-state information can be inferred from intercepted messages.

The methodology compares the proposed mechanism with plain communication, fixed permutation, and key-dependent static permutation. This allows the effect of changing the symbol mapping across episodes to be examined separately from the effect of applying a fixed transformation.

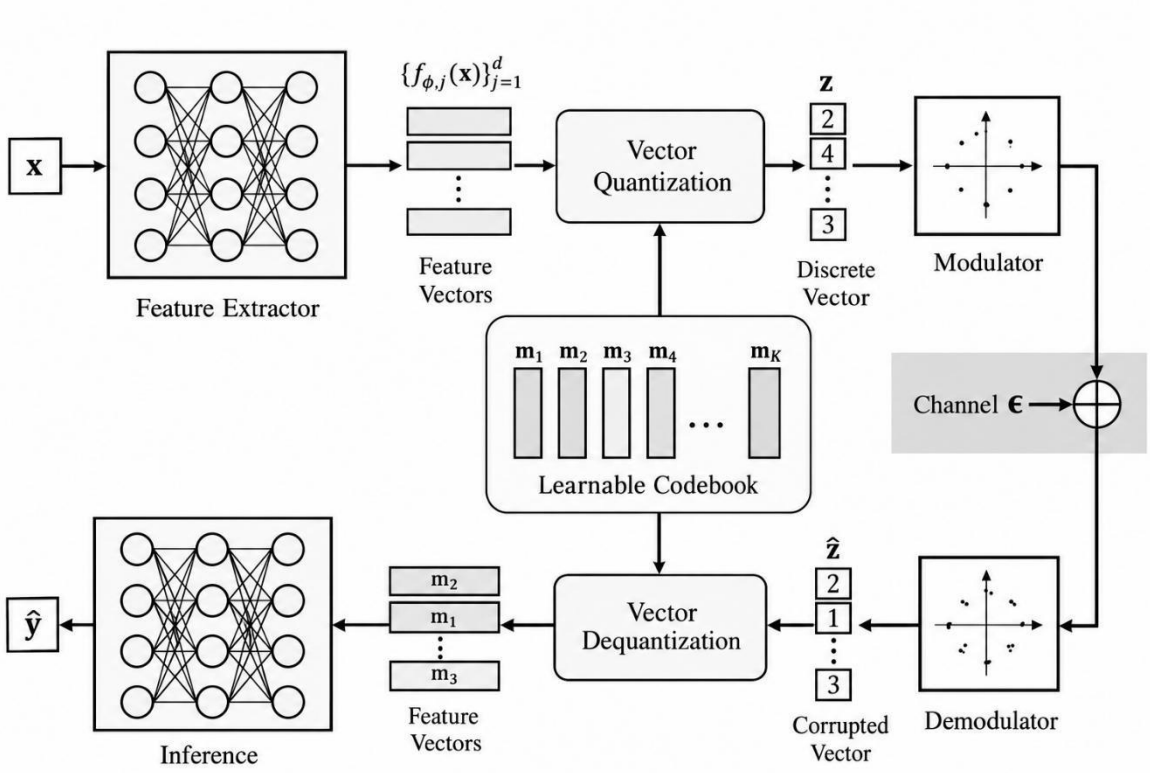


Fig 4.1 Architecture of the Learned Discrete Communication Model

4.2 OVERVIEW OF THE PROPOSED METHODOLOGY

The methodology comprises of the following major stages:

1. Input observation processing.
2. Latent representation generation using Alice's encoder.
3. Discrete symbol selection using vector quantization.
4. Generation of the episode specific permutation.
5. Remapping of the original symbol before transmission.
6. Inverse permutation at Bob's receiver.
7. Task-state prediction using the receiver decoder.
8. Task-state inference using analytical and neural eavesdroppers.
9. Comparison of the receiver and eavesdropper accuracy.

Each stage has a separate responsibility. The encoder and vector quantizer establish the original communication vocabulary, while the remapping module changes the identifiers transmitted through the communication channel. The receiver reverses this transformation so that the original learned protocol can continue to operate.

4.3 LEARNED DISCRETE COMMUNICATION MODEL

4.3.1 INPUT OBSERVATION

The communication model receives a 16-dimensional observation vector representing task related information.

Let the input observation be represented as

$$x \in \mathbb{R}^{16}$$

The corresponding task-state label belongs to one of the ten classes: $y \in \{0, 1, \ldots, 9\}$

Alice processes the observation and converts it into a latent representation. This representation contains the information used by the vector quantizer to select a discrete symbol.

4.3.2 ALICE ENCODER

Alice's encoder is a neural network that transforms the input observation into a latent vector. The encoder uses a hidden layer containing 64 units and produces a latent representation of dimension 16.

The encoder operation is expressed as $z = f_\theta(x)$

where:

- x is the input observation.
- f_θ represents the encoder network.
- θ denotes the encoder parameters.
- z is the resulting latent representation.

The encoder learns a representation that can be used by the downstream quantization and decoding stages.

4.3.3 VECTOR QUANTIZATION

Vector quantization converts the continuous latent representation into a discrete symbol selected from a finite codebook.

Let the codebook be defined as

$$C = \{e_0, e_1, \ldots, e_{15}\}$$

where each e_i represents a codebook vector. The codebook contains 16 entries.

The selected symbol is determined by the nearest codebook vector: $s = \arg\min_{i \in \{0, \ldots, 15\}} \|z - e_i\|_2^2$

Here, s denotes the original discrete symbol selected by the quantizer. The codebook index is afterwards used by the remapping mechanism.

Although the codebook contains 16 entries, not every entry is necessarily used during training or evaluation. Codebook utilisation must therefore be measured separately rather than assuming that all symbols are active.

4.3.4 BOB DECODER

After recovering the original symbol, Bob uses the corresponding codebook representation to predict the task state.

The decoder operation is represented as $\hat{y} = g_\varphi(e_s)$

where:

- g_φ represents the receiver decoder.

• e_s is the codebook vector associated with the recovered symbol.
• $\hat{y}$ is the predicted task-state label.
• φ denotes the decoder parameters.
The receiver network uses a hidden layer containing 64 units and an output layer containing ten units. The predicted class is obtained from the decoder's output.

4.4 EPISODE-DEPENDENT SYMBOL REMAPPING

4.4.1 MOTIVATION

When a symbol retains the same transmitted identifier across episodes, an eavesdropper may learn its meaning from intercepted samples. A fixed permutation changes the identifiers but preserves a stable mapping over time.
The proposed V2 Rotating mechanism changes the permutation between episodes. Consequently, a symbol identifier observed in one episode may represent a different original symbol in another episode. This is intended to reduce the usefulness of a symbol-to-state association learned from earlier communication samples.
The mechanism does not change the encoder's learned representation or the codebook itself. It operates on the discrete symbol identifier after quantization and before transmission.

4.4.2 GENERATION OF THE EPISODE-SPECIFIC PERMUTATION

Let K_secret denote the shared key and e denote the current episode index. The permutation is generated deterministically using the shared key, episode index and a domain string. The procedure can be represented as $r_e = \mathrm{SHA256}(K_secret \| e \| d)$
where:
• r_e denotes the hash-derived seed material.
• SHA256 denotes the SHA-256 hash function.
• d denotes the domain string used by the implementation.
• $\|$ denotes concatenation.
The hash-derived value is used to initialise the implementation's deterministic pseudorandom permutation-generation procedure. The resulting permutation is represented as π_e: $\{0, \ldots, 15\} \rightarrow \{0, \ldots, 15\}$
Because π_e is a permutation, each original symbol maps to one unique transmitted identifier within the episode. The same shared key and episode index allow Alice and Bob to derive matching permutations.
The exact seed conversion, domain string and pseudorandom number generator must follow the implemented code. Deterministic pseudorandom permutation generation should not be treated as equivalent to a formally analysed cryptographic construction.

4.4.3 SYMBOL TRANSMISSION

After the original symbol s is selected by the vector quantizer, Alice applies the permutation associated with episode e.
The transmitted symbol is

$$c = \pi_e(s)$$

where c represents the symbol identifier observed on the communication channel. The mapping changes according to the episode-specific permutation. Alice transmits the mapped identifier rather than the original symbol index.

4.4.4 INVERSE PERMUTATION AT BOB

Bob uses the matching permutation to recover the original symbol from the received identifier.

The inverse operation is defined as $s = \pi_e^{-1}(c)$

Since π_e is a permutation, its inverse uniquely recovers the original symbol when the correct permutation is used.

Bob then obtains the corresponding codebook vector and passes it to the decoder. This allows the original learned communication protocol to continue without requiring the receiver to learn a new codebook for every episode.

4.5 COMMUNICATION CONFIGURATIONS

Four configurations are evaluated to investigate the effect of symbol remapping.

Table 4.1 Communication configurations used in the proposed methodology

Configuration	**Description**
Plain	The original discrete symbol is transmitted without remapping.
Fixed permutation (V1)	A single permutation is applied and remains unchanged across episodes.
Key-dependent Static (V2 Static)	A permutation derived from the shared key remains constant across episodes.
Key-dependent Rotating (V2 Rotating)	The permutation depends on the shared key and episode index and changes across episodes.

Plain is the baseline for inference from untouched identifiers. V1 and V2 Static test fixed remapping. V2 Rotating tests whether changing the mapping between episodes moves eavesdropper accuracy at all.

Fair comparison needs the same task-state definitions and the same evaluation procedure in all four.

4.6 EAVESDROPPER DESIGN

Eve receives each transmitted identifier and tries to name the task state behind it. Two attackers of different construction play her role.

4.6.1 ANALYTICAL EAVESDROPPER

From intercepted training samples, the analytical attacker estimates which task-state label goes with each transmitted symbol.
For every observed symbol it takes the most likely task state given what it has seen, then applies that table to held-out samples.
Under plain transmission or any fixed mapping, a stable symbol-to-state relationship gives this attacker a consistent table to learn. Under V2 Rotating the relationship shifts from episode to episode, and a table built from past episodes loses its value.
It is a deliberately simple baseline for how much the identifiers themselves give away.

4.6.2 NEURAL EAVESDROPPER

The neural attacker is a multilayer perceptron that classifies intercepted symbols, each fed in as a one-hot vector of length 16.
Two hidden layers of 128 units each sit in front of a ten-unit output layer.
It computes $\hat{y}_Eve = h_\psi(c)$
where h_ψ denotes the attacker network and ψ represents its parameters. The predicted label $\hat{y}_Eve$ is compared with the actual task-state label.
It is trained on intercepted samples and tested on held-out messages. With a single one-hot symbol as input, the best this network can do is approximate the same symbol-to-label table the analytical attacker builds by counting, so its score works as a check on that table. Low accuracy here does not make the channel secure against every attacker. A stronger Eve could use episode information, temporal patterns, auxiliary observations or features this model never sees.

4.7 EVALUATION METRICS

4.7.1 RECEIVER ACCURACY

Bob's accuracy is the fraction of evaluated examples for which he names the correct task state.
It is computed as

$$A_Bob = (1/N) \Sigma_{i=1}^{N} I(\hat{y}_i = y_i) \times 100\%$$

with N the number of evaluated examples and I the indicator function, 1 for a correct prediction and 0 otherwise.
High values mean Bob reads the communicated task information correctly under the tested conditions.

4.7.2 EAVESDROPPER ACCURACY

Eve's accuracy counts the samples on which she names the correct task state.
It is computed as

$$A_Eve = (1/N) \Sigma_{i=1}^{N} I(\hat{y}_Eve,_i = y_i) \times 100\%$$

Analytical and neural attackers are scored separately. Tracking both across configurations shows whether episode-dependent remapping lowers task-state inference.

4.7.3 RANDOM-GUESSING REFERENCE

With ten equally likely task states, uniform guessing is expected to score

$$A_{random} = (1/10) \times 100\% = 10\%$$

Eve's numbers are read against this line. Imbalanced labels move it: a classifier that always outputs the majority class can beat 10% without learning anything about the symbols. The label distribution has to be checked before an attacker's score is set against the guessing reference.

4.8 ALGORITHM FOR V2 ROTATING

Algorithm 4.1. Episode-dependent symbol remapping Input:
Observation x, shared key K_secret, episode index e.
Output: Receiver prediction $\hat{y}$ and transmitted symbol c.

1. Alice receives observation x.
2. Compute the latent representation $z = f_\theta(x)$.
3. Select the nearest codebook entry and obtain symbol s.
4. Derive the deterministic seed material from the shared key, episode index, and domain string.
5. Generate the episode-specific permutation π_e.
6. Compute the transmitted symbol $c = \pi_e(s)$.
7. Transmit c through the communication channel.
8. Bob derives the matching permutation using the shared key and episode index.
9. Recover the original symbol $s = \pi_e^{-1}(c)$.
10. Obtain the corresponding codebook vector.
11. Compute the receiver prediction $\hat{y} = g_\varphi(e_s)$.
12. Eve observes c and attempts to infer the task-state label.
13. Evaluate receiver and eavesdropper predictions using the selected metrics.

4.9 METHODOLOGY SUMMARY

Learned discrete communication and deterministic, episode-dependent remapping are combined in one pipeline. Alice quantizes an observation to a symbol, relabels it with the current episode's permutation and sends it. Bob inverts the permutation and decodes as usual. Both eavesdroppers then attempt task-state inference from the intercepted symbols. Running plain transmission, V1, V2 Static and V2 Rotating side by side gives an empirical handle on how mapping stability affects an attacker, with everything else in the pipeline held fixed. All of this is evaluated inside one simulation and one threat model. No formal cryptographic guarantee follows from it. Chapter 5 covers the software, model architecture, training setup and experimental workflow.

Chapter 5

System Implementation

5.1 INTRODUCTION

The implementation phase converts the proposed system design into a working softwarebased communication environment. The system is developed to study the behaviour of learned discrete communication under different symbol-mapping configurations and to evaluate the ability of an external eavesdropper to infer task states from intercepted messages.

The implementation consists of a sender model, a vector quantization module, a symbolremapping mechanism, a receiver model, and two eavesdropper models. The sender and receiver use neural networks to process observations and recover task-related information, while the remapping module modifies the transmitted symbol identifiers according to the selected communication configuration.

The experiments are conducted in a simulated environment, avoiding the need for physical robots or specialised communication hardware. This makes it possible to evaluate the communication mechanism under controlled conditions and compare the performance of the different configurations using consistent metrics.

Table 5.1 Software and development components

Component	**Technology**	**Purpose**
Programming language	Python	Implementation of the communication pipeline and evaluation scripts
Deep learning framework	PyTorch	Construction and training of the neural models
Numerical processing	NumPy	Array operations and numerical calculations
Hashing	SHA-256	Deterministic generation of seed material
Pseudorandom generation	Python random	Generation of deterministic permutations from derived seeds

Data handling	Python data structures and dataset utilities	Storage and processing of observations, symbols, and labels
Evaluation	Python scripts	Calculation of receiver and eavesdropper accuracy
Execution environment	Software simulation	Controlled experimentation without physical robotic hardware

The precise software versions should be recorded from the final execution environment used to produce the reported results.

5.2 DEVELOPMENT ENVIRONMENT

The project is implemented using Python and PyTorch. PyTorch supports the construction and training of neural networks, the processing of latent representations, and the implementation of the analytical and neural attacker models. Additional Python libraries support deterministic seed generation, permutation handling, data processing, and performance evaluation.

5.3 DATASET AND SIMULATION ENVIRONMENT

Ten task states, each observed as a 16-dimensional feature vector, define the simulated task. Alice maps an observation to a latent vector and then to a discrete symbol.

Sixteen codebook entries are available. Earlier analysis showed that only ten of them were used in the evaluated protocol, which equals the number of task states, so the effective vocabulary is smaller than the codebook suggests, and the six idle entries change how that vocabulary should be read.

Environmental complexity was stripped out on purpose. Obstacles were disabled in the main Grid World experiment so that the only thing being measured is the link between transmitted symbols and task-state inference.

Table 5.2 Main simulation characteristics

Parameter	**Configuration**
Environment	Simulated Grid World
Grid dimensions	7 × 7
Number of task states	10
Input observation dimension	16
Latent representation dimension	16
Codebook size	16 entries
Sender encoder hidden layer	64 units

Receiver decoder hidden layer	64 units
Receiver output dimension	10
Obstacles in the main experiment	Disabled

5.4 IMPLEMENTATION OF THE SENDER MODEL

5.4.1 OBSERVATION ENCODING

The sender, Alice, is implemented as a feed-forward neural network. It accepts the 16dimensional observation vector and transforms it into latent representation of dimension 16. The encoder uses a hidden layer with 64 units. Its output is passed to the vector quantizer, which selects the codebook entry closest to the latent representation.

The encoder operation is represented as $z = f_\theta(x)$
where x is the input observation, f_θ denotes the encoder, and z represents the latent vector. The selected symbol is then obtained through vector quantization. This symbol forms the input to the communication-remapping stage.

5.4.2 VECTOR QUANTIZATION MODULE

The vector quantization module maintains a codebook of 16 entries. Each entry represents a learned latent vector associated with a discrete symbol identifier.
For an encoder output z, the module selects the nearest codebook vector according to the implemented distance function. The selected index is the original symbol s.
The distinction between the symbol index and its codebook vector is important. The index identifies the selected entry, while the vector represents the corresponding learned latent representation. The remapping mechanism operates on the discrete identifier rather than modifying the codebook vector itself.

```
======================================================================
LEARNED STATE → SYMBOL MAPPING
======================================================================
NORTH        → symbol  5 | purity = 100.00% | samples = 214
SOUTH        → symbol  8 | purity = 100.00% | samples = 233
EAST         → symbol 10 | purity = 100.00% | samples = 215
WEST         → symbol  4 | purity = 100.00% | samples = 198
NORTHEAST    → symbol 12 | purity = 100.00% | samples = 475
NORTHWEST    → symbol  2 | purity = 100.00% | samples = 553
SOUTHEAST    → symbol  3 | purity = 100.00% | samples = 484
SOUTHWEST    → symbol 15 | purity = 100.00% | samples = 552
NEAR         → symbol  7 | purity = 100.00% | samples = 330
FAR          → symbol  9 | purity = 100.00% | samples = 1746

======================================================================
LEARNED SYMBOL → STATE MAPPING
======================================================================
Symbol  0 → UNUSED
Symbol  1 → UNUSED
Symbol  2 → NORTHWEST    | purity = 100.00% | uses = 553
Symbol  3 → SOUTHEAST    | purity = 100.00% | uses = 484
Symbol  4 → WEST         | purity = 100.00% | uses = 198
Symbol  5 → NORTH        | purity = 100.00% | uses = 214
Symbol  6 → UNUSED
Symbol  7 → NEAR         | purity = 100.00% | uses = 330
Symbol  8 → SOUTH        | purity = 100.00% | uses = 233
Symbol  9 → FAR          | purity = 100.00% | uses = 1746
Symbol 10 → EAST         | purity = 100.00% | uses = 215
Symbol 11 → UNUSED
Symbol 12 → NORTHEAST    | purity = 100.00% | uses = 475
Symbol 13 → UNUSED
Symbol 14 → UNUSED
Symbol 15 → SOUTHWEST    | purity = 100.00% | uses = 552
```

Figure 5.1 Symbol-State mapping after vector quantization of Emergent Communication

5.5 IMPLEMENTATION OF THE REMAPPING MODULE

The remapping module implements the communication configurations used in the experiments. Depending on the selected configuration, the original symbol is either transmitted directly or transformed using a permutation.

5.5.1 PLAIN COMMUNICATION

In the plain configuration, Alice transmits the original symbol without applying a permutation. c = s

Here, s denotes the symbol selected by the vector quantizer and c denotes the transmitted identifier. This configuration establishes the baseline against which the remapping methods are evaluated.

```
EVE'S LEARNED SYMBOL -> STATE MAPPING
============================================
Symbol  1 -> State 4
Symbol  2 -> State 0
Symbol  3 -> State 7
Symbol  6 -> State 8
Symbol  7 -> State 5
Symbol  8 -> State 2
Symbol  9 -> State 3
Symbol 11 -> State 6
Symbol 12 -> State 1
Symbol 13 -> State 9

ANALYTICAL EVE RESULTS
============================================
Messages used to learn mapping : 1000
Messages used for testing      : 1000
Correct predictions             : 1000
Eve accuracy                    : 100.00%
Random chance accuracy          : 10.00%
```

Figure 5.2 Analytical Eve's attack on plain communication

5.5.2 FIXED PERMUTATION

In the fixed configuration, a single permutation is applied to the codebook indices. The same mapping is retained across communication episodes. $c = \pi(s)$

Although the transmitted identifiers differ from the original symbol identifiers, the mapping remains stable. Consequently, an attacker who learns the transformed association may continue to use it in later episodes.

```
V1 COMMUNICATION EXAMPLES
============================================================
State  9 | Original symbol 13 | Transmitted symbol 11 | Recovered symbol 13
State  7 | Original symbol  3 | Transmitted symbol  1 | Recovered symbol  3
State  2 | Original symbol  8 | Transmitted symbol 14 | Recovered symbol  8
State  6 | Original symbol 11 | Transmitted symbol 12 | Recovered symbol 11
State  6 | Original symbol 11 | Transmitted symbol 12 | Recovered symbol 11
State  8 | Original symbol  6 | Transmitted symbol 10 | Recovered symbol  6
State  9 | Original symbol 13 | Transmitted symbol 11 | Recovered symbol 13
State  1 | Original symbol 12 | Transmitted symbol  7 | Recovered symbol 12
State  9 | Original symbol 13 | Transmitted symbol 11 | Recovered symbol 13
State  4 | Original symbol  1 | Transmitted symbol 13 | Recovered symbol  1

V1 COMMUNICATION RESULTS
============================================================
Test messages          : 2000
Correct Bob predictions: 2000
Bob accuracy           : 100.00%
Symbol round-trip errors: 0
```

Figure 5.3.1 Fixed mapping (V1) communication results

```
EVE'S INFERRED V1 SYMBOL -> STATE MAPPING
================================================
Transmitted symbol  1 -> State 7
Transmitted symbol  3 -> State 5
Transmitted symbol  7 -> State 1
Transmitted symbol  8 -> State 0
Transmitted symbol 10 -> State 8
Transmitted symbol 11 -> State 9
Transmitted symbol 12 -> State 6
Transmitted symbol 13 -> State 4
Transmitted symbol 14 -> State 2
Transmitted symbol 15 -> State 3

ANALYTICAL EVE AGAINST V1
================================================
Messages used to learn mapping : 2500
Messages used for testing      : 2500
Correct predictions             : 2500
Unknown transmitted symbols     : 0
Eve accuracy                    : 100.00%
Random chance accuracy          : 10.00%

RESULT: Eve has learned significant information from the static V1 mapping.
```

Figure 5.3.2 Analytical Eve Attack on Fixed Mapping (V1) communication

5.5.3 KEY-DEPENDENT STATIC PERMUTATION

Here the permutation is derived from the shared key and then held for all episodes.
Alice transmits $c = \pi_K(s)$
where π_K is the permutation fixed by the shared key. Key-dependent remapping is tested here without any change between episodes, which separates the role of the key from the role of rotation.

5.5.4 V2 ROTATING PERMUTATION

For V2 Rotating the permutation belongs to the episode. SHA-256 turns the shared key and episode index into deterministic seed material, and that material seeds the implemented pseudorandom permutation generator. Alice transmits $c = \pi_e(s)$
where π_e is the permutation for episode e.
Alice and Bob arrive at the same permutation whenever key material, episode information and generation parameters match. Bob can then undo the transformation, and the learned protocol stays as it was trained.
Concretely, the code hashes to a seed and hands it to Python's random module. That module is a Mersenne Twister, built for simulation rather than for resisting an adversary who studies its output, and the security of the full mechanism has not been analysed. It is not a cryptographic construction.

```
V2 STATIC ROUND-TRIP TEST
============================================================
Symbol  0 -> Transmitted 14 -> Recovered  0
Symbol  1 -> Transmitted  5 -> Recovered  1
Symbol  2 -> Transmitted 10 -> Recovered  2
Symbol  3 -> Transmitted  8 -> Recovered  3
Symbol  4 -> Transmitted  0 -> Recovered  4
Symbol  5 -> Transmitted 13 -> Recovered  5
Symbol  6 -> Transmitted  7 -> Recovered  6
Symbol  7 -> Transmitted  6 -> Recovered  7
Symbol  8 -> Transmitted  4 -> Recovered  8
Symbol  9 -> Transmitted 15 -> Recovered  9
Symbol 10 -> Transmitted 11 -> Recovered 10
Symbol 11 -> Transmitted  9 -> Recovered 11
Symbol 12 -> Transmitted  1 -> Recovered 12
Symbol 13 -> Transmitted  2 -> Recovered 13
Symbol 14 -> Transmitted  3 -> Recovered 14
Symbol 15 -> Transmitted 12 -> Recovered 15

V2 static round-trip test PASSED

V2 ROTATING ROUND-TRIP TEST
============================================================
Episode  0 | Original symbol  7 | Transmitted symbol  6 | Recovered symbol  7
Episode  1 | Original symbol  7 | Transmitted symbol  6 | Recovered symbol  7
Episode  2 | Original symbol  7 | Transmitted symbol  6 | Recovered symbol  7
Episode  3 | Original symbol  7 | Transmitted symbol 10 | Recovered symbol  7
Episode  4 | Original symbol  7 | Transmitted symbol 15 | Recovered symbol  7
Episode  5 | Original symbol  7 | Transmitted symbol 15 | Recovered symbol  7
Episode  6 | Original symbol  7 | Transmitted symbol  9 | Recovered symbol  7
Episode  7 | Original symbol  7 | Transmitted symbol 11 | Recovered symbol  7
Episode  8 | Original symbol  7 | Transmitted symbol  4 | Recovered symbol  7
Episode  9 | Original symbol  7 | Transmitted symbol  0 | Recovered symbol  7

V2 rotating round-trip test PASSED
```

Figure 5.4 V2 communication results

```
STATIC VS ROTATING COMPARISON
============================================================

Static mode:
------------------------------
Episode 0 -> Transmitted symbol 6
Episode 1 -> Transmitted symbol 6
Episode 2 -> Transmitted symbol 6
Episode 3 -> Transmitted symbol 6
Episode 4 -> Transmitted symbol 6

Rotating mode:
------------------------------
Episode 0 -> Transmitted symbol 6
Episode 1 -> Transmitted symbol 6
Episode 2 -> Transmitted symbol 6
Episode 3 -> Transmitted symbol 10
Episode 4 -> Transmitted symbol 15
```

Figure 5.5 Comparison of V2 static vs V2 rotating communication mode

5.6 IMPLEMENTATION OF THE RECEIVER MODEL

Bob is the intended receiver. Under plain transmission he processes the received symbol directly. Under any remapped configuration he first rebuilds the permutation and applies its inverse.

Recovery is $s = \pi_e^{-1}(c)$

for the rotating configuration, with the matching inverse used for the other permutationbased configurations.

With the original symbol back, Bob fetches its codebook vector and runs his decoder, one hidden layer of 64 units and an output layer of ten, to predict the task state.

Remapping changes the transmitted identifier, never the symbol that reaches the decoder. Without that property, Bob's accuracy could not be compared across configurations, since a drop in his score would then mix decoding errors with remapping errors.

5.7 IMPLEMENTATION OF THE EAVESDROPPERS

Two attackers, one counting-based and one learned, measure task-state inference from intercepted messages.

5.7.1 ANALYTICAL EAVESDROPPER

Using intercepted training samples, the analytical eavesdropper builds a table from transmitted symbols to task-state labels and applies it to held-out samples.

It answers a simple question: do the identifiers keep a stable association with task states?

How well it does depends on sample count, mapping consistency and the task-state distribution. With one symbol per message, its learned table has at most 16 rows.

In V2 Rotating, the original symbol behind an identifier can change from episode to episode. Without the current permutation, a table built on earlier episodes stops paying off.

5.7.2 NEURAL EAVESDROPPER

The neural eavesdropper is a multilayer perceptron. Each transmitted symbol enters as a 16dimensional one-hot vector, passes through two hidden layers of 128 units each, and leaves through a ten-unit output layer.

Table 5.3 Neural eavesdropper architecture

Layer	**Configuration**
Input	16-dimensional one-hot vector
Hidden layer 1	128 units
Hidden layer 2	128 units
Output layer	10 units
Prediction	Task-state class

The neural attacker learns a classification function from the intercepted training samples. During evaluation, its predictions are compared with the actual task-state labels.
The purpose of this model is to measure the performance of a learned classifier under the specified observation constraints. Its accuracy should not be interpreted as a guarantee about the performance of every possible attacker.

```
======================================================================
ANALYTICAL EVE - V2 STATIC
======================================================================

Messages used to learn mapping : 2500
Messages used for testing      : 2500
Unique transmitted symbols     : 10/16

EVE'S LEARNED TRANSMITTED SYMBOL -> STATE MAPPING
----------------------------------------------------------
Transmitted symbol  1 -> State 1
Transmitted symbol  2 -> State 9
Transmitted symbol  4 -> State 2
Transmitted symbol  5 -> State 4
Transmitted symbol  6 -> State 5
Transmitted symbol  7 -> State 8
Transmitted symbol  8 -> State 7
Transmitted symbol  9 -> State 6
Transmitted symbol 10 -> State 0
Transmitted symbol 15 -> State 3

ANALYTICAL EVE RESULTS
----------------------------------------------------------
Eve accuracy                   : 100.00%
Random chance accuracy         : 10.00%
```

Figure 5.6.1 Analytical Eve attack on V2-static communication mode

```
======================================================================
ANALYTICAL EVE - V2 ROTATING
======================================================================

Messages used to learn mapping : 2500
Messages used for testing      : 2500
Unique transmitted symbols     : 16/16

EVE'S LEARNED TRANSMITTED SYMBOL -> STATE MAPPING
----------------------------------------------------------
Transmitted symbol  0 -> State 6
Transmitted symbol  1 -> State 0
Transmitted symbol  2 -> State 7
Transmitted symbol  3 -> State 8
Transmitted symbol  4 -> State 8
Transmitted symbol  5 -> State 5
Transmitted symbol  6 -> State 4
Transmitted symbol  7 -> State 8
Transmitted symbol  8 -> State 9
Transmitted symbol  9 -> State 7
Transmitted symbol 10 -> State 8
Transmitted symbol 11 -> State 1
Transmitted symbol 12 -> State 4
Transmitted symbol 13 -> State 8
Transmitted symbol 14 -> State 1
Transmitted symbol 15 -> State 4

ANALYTICAL EVE RESULTS
----------------------------------------------------------
Eve accuracy                   : 10.28%
Random chance accuracy         : 10.00%
```

Figure 5.6.2 Analytical Eve attack on V2-rotating communication mode

5.8 EXPERIMENTAL WORKFLOW

All configurations go through the same sequence:

1. Prepare task observations and their state labels.
2. Encode each observation with Alice's encoder.
3. Quantize to obtain the original symbol.
4. Apply the transformation for the active configuration.
5. Transmit the resulting identifier.
6. Let Bob recover the original symbol and predict the task state.
7. Pass the transmitted identifiers to both eavesdroppers.
8. Score Bob's and Eve's predictions.
9. Compare the accuracies across configurations.

Plain, fixed, static and rotating runs share this workflow. For the comparison to be fair, train/test separation, task-state definitions and scoring must not vary between them, and Eve's information limits must hold while Bob still gets the correct inverse mapping.

5.9 IMPLEMENTATION CHALLENGES

Four implementation details decide how far the results can be trusted. Matching permutations come first. If the key, episode index, domain string, seed conversion or generator differ at all between Alice and Bob, Bob decodes the wrong symbol.

Unused codebook entries come next. Some entries are never selected by the trained encoder, and measured utilisation, not the nominal 16, is what should be reported. Earlier analysis found ten of the 16 entries in use.

Data splits matter for both attackers. Without consistent separation of training and test samples, measured inference accuracy stops describing performance on unseen data.

Script disagreement is the open problem. Preliminary experiments produced differing accuracy figures, and the final report should carry only numbers produced by one verified, consistently executed evaluation pipeline.

5.10 CHAPTER SUMMARY

Chapter 5 covered the development environment, simulation settings, sender encoder, vector quantizer, remapping module, receiver decoder and both eavesdroppers, along with the shared workflow that drives them.

Four configurations run through one shared workflow that scores Bob's accuracy and Eve's inference. V2 Rotating adds the episode-dependent permutation, and Bob removes it with the inverse.

Chapter 6 reports what the experiments produced, compares the configurations and sets out where the method falls short. Figures there are marked preliminary wherever the evaluation scripts have not yet been reconciled.

Chapter 6

Results And Discussion

6.1 INTRODUCTION

Does episode-dependent symbol remapping change task-state inference? Four configurations were run to find out: plain transmission, fixed permutation, key-dependent static permutation and the key-dependent episode-varying permutation, V2 Rotating.

Two things are measured. First, does Bob still recover the communicated task information? Second, can Eve, listening on the channel, infer the task state from intercepted symbols? Eve is played by an analytical attacker, which estimates symbol-to-state associations, and by a neural attacker that learns a classifier from intercepted samples. Across the four configurations, their scores show whether changing the mapping between episodes matters, and Bob's score shows what that change costs the intended receiver.

Every number in this chapter is preliminary. Some of the evaluation scripts disagree with one another, and until they are reconciled the figures are observations from the available runs, not validated performance measurements.

6.2 EXPERIMENTAL CONFIGURATION

Experiments ran in a simulated 7 × 7 Grid World. Ten task states are defined, each presented as a 16-dimensional observation.

Alice's encoder maps each observation to a latent vector that is quantized against a 16-entry codebook. Bob predicts the task state from the recovered symbol. Eve receives the transmitted identifier and predicts the label with no access to the episode's permutation.

Table 6.1 Experimental configuration

Parameter	**Configuration**
Simulation environment	GridWorld
Grid dimensions	7 × 7
Number of task states	10
Input dimension	16
Latent dimension	16
Codebook size	16
Encoder hidden layer	64 units
Receiver hidden layer	64 units
Neural attacker hidden layers	Two layers of 128 units each
Number of communication configurations	4
Eavesdropper models	Analytical and neural

The random-guessing reference for a balanced ten-class task is 10%. This value is used to contextualise the observed eavesdropper accuracy, provided that the task-state distribution supports that comparison.

6.3 RECEIVER COMMUNICATION ACCURACY

Receiver accuracy measures the proportion of test examples for which Bob predicts the correct task state after receiving and decoding the transmitted symbol.

In the reported preliminary experiments, receiver accuracy reached 100% under the evaluated remapping configurations. This indicates that the inverse-mapping operation can preserve communication functionality when Bob uses the correct episode-specific permutation.

Table 6.2 Receiver accuracy across communication configurations

Configuration	**Reported receiver accuracy**
Plain communication	100%
Fixed permutation (V1)	100%
Key-dependent static permutation (V2 Static)	100%
Key-dependent rotating permutation (V2 Rotating)	100%

```
======================================================================
END-TO-END EXPERIMENT SUMMARY
======================================================================

plain        | Task success: 100.00% | Communication: 100.00% | Avg steps: 3.50

v1           | Task success: 100.00% | Communication: 100.00% | Avg steps: 3.50

v2_static    | Task success: 100.00% | Communication: 100.00% | Avg steps: 3.50

v2_rotating  | Task success: 100.00% | Communication: 100.00% | Avg steps: 3.50
```

Figure 6.1 Accuracy (Bob’s decoding accuracy) for communication modes

At minimum, then, the intended receiver recovered the task information in the reported evaluation. Figures for plain, V1 and V2 Static still have to be produced on the same heldout set with the same procedure before Table 6.2 can be read as a comparison.

High receiver accuracy says nothing about eavesdropping. It is Bob's ability to decode, and it has to be read separately from Eve's inference accuracy.

6.4 EAVESDROPPER ACCURACY

Eve's accuracy is how often she names the correct task-state label for an intercepted symbol. Lower scores mean a less successful attacker under these specific conditions, which is a weaker statement than saying no task information leaks.

Table 6.3 gives the preliminary results of the robot-specific evaluation.

Table 6.3 Preliminary eavesdropper accuracy

Configuration	**Analytical eavesdropper**	**Neural eavesdropper**	**Random reference**
Plain communication	100.00%	100.00%	10.00%
Fixed permutation (V1)	100.00%	100.00%	10.00%
Key-dependent static (V2 Static)	100.00%	100.00%	10.00%
Key-dependent rotating (V2 Rotating)	9.32%	9.36%	10.00%

```
######################################################################
ROBOT EVE EXPERIMENT: v2_rotating
######################################################################

Analytical Eve accuracy: 9.32%

Neural Eve training:
Epoch  1/30 | Loss: 2.3053 | Train accuracy: 9.76%
Epoch  5/30 | Loss: 2.3007 | Train accuracy: 11.24%
Epoch 10/30 | Loss: 2.2975 | Train accuracy: 12.00%
Epoch 15/30 | Loss: 2.2946 | Train accuracy: 12.52%
Epoch 20/30 | Loss: 2.2913 | Train accuracy: 13.24%
Epoch 25/30 | Loss: 2.2878 | Train accuracy: 13.48%
Epoch 30/30 | Loss: 2.2844 | Train accuracy: 13.56%

Neural Eve accuracy: 9.36%

======================================================================
ROBOT EVE ATTACK SUMMARY
======================================================================

Mode            | Analytical Eve  | Neural Eve      | Random chance
----------------------------------------------------------------------
plain           |         100.00% |         100.00% |         10.00%
v1              |         100.00% |         100.00% |         10.00%
v2_static       |         100.00% |         100.00% |         10.00%
v2_rotating     |           9.32% |           9.36% |         10.00%
```

Figure 6.2 Comparison of Eve attacks on different communication modes

6.4.1 PLAIN COMMUNICATION

Plain transmission sends the original symbol untouched. Both eavesdroppers scored 100%. Exactly this would follow from a stable one-to-one link between symbols and task states. Ten active codebook entries against ten task states leave room for a clean lookup table, and an attacker with labelled samples only has to copy it down.

Confirmation on the final split with a verified attacker is still needed before that reading can stand as the baseline result.

6.4.2 FIXED PERMUTATION

V1 applies one transformation to the symbol vocabulary and never changes it.

Both attackers again scored 100%. Renaming the identifiers with a fixed permutation leaves the relationship between original and transmitted symbols exactly as stable as before.

Given enough labelled observations, Eve simply learns the renamed table. A fixed permutation offers no real protection against an observer able to learn the mapping. Relabelling moves the problem without removing it: Eve's lookup table keeps the same number of rows, only with different identifiers in the first column.

6.4.3 KEY-DEPENDENT STATIC PERMUTATION

V2 Static derives its permutation from the shared key and then holds it for every episode. Both eavesdroppers scored 100% here too. Under the tested conditions, a key-dependent but static mapping did not stop either attacker from learning which transmitted symbol meant which task state.

Having a shared key is therefore not enough on its own. What decides the outcome is whether the transmitted mapping holds still and whether Eve can get labelled samples.

6.4.4 V2 ROTATING

V2 Rotating derives its permutation from the shared key and the episode index, so the mapping changes from one episode to the next.

Analytical and neural eavesdroppers scored 9.32% and 9.36% respectively. Both sit just under the 10% expected from uniform guessing over ten balanced states.

That fits a learned table carrying nothing useful from one episode into the next. The attackers also land almost together. With a single one-hot symbol as input, the MLP can at best reproduce the analytical attacker's counting table, so a 0.04-point gap between them is unsurprising.

Near-chance accuracy is not proof that nothing leaks. Any conclusion is limited to these two attackers, this data and these observation limits, and without the test-set size and run-to-run spread nobody can say whether 9.32% is meaningfully below 10%.

6.5 COMPARISON OF STATIC AND ROTATING REMAPPING

The comparison that matters is static mapping against V2 Rotating.

Plain sends the original symbol. V1 and V2 Static apply a permutation that never changes. Once Eve has learned the renamed table, the link between identifiers and task states keeps paying off for her in every later episode.

Rotation breaks that link between episodes. An identifier seen in one episode need not point to the same original symbol in the next, and an attacker that only knows how to learn a stable symbol-to-state table has nothing stable to learn.

Table 6.4 Qualitative comparison of remapping behaviour

Property	**Plain**	**Fixed**	**V2 Static**	**V2 Rotating**
Original symbol transmitted directly	Yes	No	No	No
Mapping changes across episodes	No	No	No	Yes
Requires inverse mapping at receiver	No	Yes	Yes	Yes
Stable transmitted mapping	Yes	Yes	Yes	No
Designed to disrupt stable symbol associations	No	Limited	Limited	Yes

Table 6.4 describes how each configuration is meant to behave. It is not a security ranking, and real eavesdropper performance depends on the implementation, the attacker, the training data and the test protocol.

6.6 COMPARISON WITH THE RANDOM-GUESSING REFERENCE

Uniform random guessing over ten balanced states is expected to score A_random

$$= (1/10) \times 100\% = 10\%.$$

At 9.32% and 9.36%, the preliminary V2 Rotating results sit next to this value. In that experiment, neither attacker did substantially better than guessing, and both scores sit less than one percentage point from 10%.

Reading the comparison needs care. If the labels are imbalanced, a majority-class classifier beats 10% without learning anything about the symbols. Eve's accuracy should be reported next to the class distribution, together with balanced accuracy and per-class results. Exact-match accuracy also misses partial leakage. Even an attacker who rarely names the precise state could still narrow it down, and the guessing reference would not register that.

6.7 LIMITATIONS OF THE EXPERIMENTAL RESULTS

Five limitations bound these findings.

Standalone attacker evaluations and the combined benchmark disagree. Reconciling them needs one dataset split, one set of label definitions and one evaluation procedure.

Only single-symbol classification was tested. Attackers that read longer message sequences, use episode context or auxiliary observations, or know how the keys are generated were not.

Simplicity of the environment matters too: a simulated GridWorld with obstacles switched off for the main evaluation. More complex robotic settings may behave differently.

Permutation generation is deterministic and keyed, but its security has not been formally analysed, and it cannot stand in for standard encryption, which comes with security analysis this scheme lacks.

Bob's and Eve's results still have to be reproduced across multiple random seeds. Reporting the spread over seeds would show whether the gaps between configurations survive changes in initialisation and sampled data.

6.8 CHAPTER SUMMARY

Chapter 6 tested the communication system under plain, fixed, key-dependent static and episode-dependent rotating remapping. Plain, V1 and V2 Static gave both eavesdroppers 100% in the robot-specific runs. Under V2 Rotating they dropped to 9.32% and 9.36%, close to the 10% guessing reference, and Bob's accuracy in the matching preliminary run was 100%.

That pattern fits the hypothesis that changing the mapping between episodes takes away the value of a stable symbol-to-state table. It is not yet conclusive, because the scripts still disagree, and it shows neither formal cryptographic security nor resistance to stronger attackers. Chapter 7 draws the conclusions, lists what the project contributed and sets out future work.

Chapter 7 Conclusion And Future Work

7.1 CONCLUSION

Stable symbol-to-state associations make learned discrete robot communication readable by anyone who collects enough samples. This project tested episode-dependent symbol remapping as a way to cut that kind of task-state inference.

A learned pipeline was built from a neural encoder, a vector quantizer with a discrete codebook, and a receiver decoder, and V2 Rotating sits on top of it, changing the transmitted identifier every episode using a shared key and the episode index. Alice permutes before sending. Bob inverts the permutation, gets the original symbol back and decodes as normal. Plain transmission, a fixed permutation, a key-dependent static permutation and the keydependent episode-varying permutation were all run, and analytical and neural eavesdroppers measured how much task-state information the intercepted symbols gave away.

In the preliminary robot-specific evaluation, rotation brought the analytical and neural eavesdroppers to 9.32% and 9.36%, against a 10% reference for uniform guessing over ten balanced states. Bob stayed at 100%. For these two attackers, changing the mapping between episodes appears to strip a stable symbol-to-state table of its value.

Those numbers still need verifying, since the evaluation scripts disagree. They are preliminary evidence, not a demonstration of security. The method carries no formal cryptographic guarantee and has not been tested against every eavesdropping strategy. What the project leaves behind is an experimental setup for studying how learned discrete codes, shifting symbol mappings, receiver reliability and outside inference interact.

7.2 ACHIEVEMENTS OF THE PROJECT

Six things were built and tested:

1. Learned communication pipeline: a model that turns task observations into discrete symbols and reconstructs task-state predictions at the receiver.
2. Vector-quantized representation: a 16-entry codebook through which all communication passes as discrete symbols.
3. Episode-dependent remapping: a key-dependent permutation that changes the transmitted identifiers between episodes.
4. Receiver-side recovery: an inverse permutation that lets Bob get the original symbol back.
5. Eavesdropper evaluation: analytical and neural attackers that measure task-state inference from intercepted symbols.
6. Comparative evaluation: plain, fixed, static keyed and rotating configurations, run to isolate how mapping stability affects attacker performance.

Together these form the implementation and evaluation setup. How well the mechanism works has to be judged from the final, reconciled results, not from the fact that the setup exists.

7.3 LIMITATIONS

Running remapping per episode inside a simulated system cost the receiver nothing in the reported run. Four limits remain on what that shows.

The environment is simple, with no physical robots, no complex navigation, and obstacles switched off in the main experiment. Real deployments could give different numbers. Both eavesdroppers infer the task state from intercepted identifiers alone. An attacker that exploits episode boundaries, message sequences, auxiliary observations or other information outside the threat model could do better.

Permutations come from deterministic, key-dependent seeding whose security has never been formally analysed. It is not a replacement for established encryption.

The evaluation scripts disagree on some figures. Until one consistent test procedure and repeated runs exist, the relative standing of the configurations is provisional.

7.4 FUTURE WORK

Five extensions follow from these limits.

7.4.1 EVALUATION AGAINST STRONGER EAVESDROPPERS

Eve should next get more to work with: sequences of intercepted messages, episode context, knowledge of the environment. Running those attackers would show whether the drop in classification accuracy holds once she has richer information.

7.4.2 EXTENSION TO MORE COMPLEX ROBOTIC ENVIRONMENTS

Obstacles, longer navigation tasks and richer interactions between agents are the obvious additions to the environment. Testing there would show whether the behaviour seen in the 7 × 7 GridWorld carries over.

7.4.3 REPRODUCIBILITY AND STATISTICAL EVALUATION

Every configuration should be rerun across several random seeds with one dataset split, one training procedure and one attacker setup. Mean accuracy, run-to-run spread and further classification metrics would put the comparison on firmer footing.

7.4.4 IMPROVED KEY MANAGEMENT

Key management and permutation generation need strengthening, starting with a cryptographically secure generator in place of Python's random module, and the result should be checked against established security requirements before any confidentiality claim is made. Running remapping alongside standard encryption would also show what each contributes and what each costs.

7.4.5 EXTENSION TO MULTI-AGENT COMMUNICATION

So far there is one sender, one intended receiver and one eavesdropper. Several robots, with different pairs exchanging messages under episode-dependent mappings, would raise questions this setup never had to answer: synchronisation, key distribution, communication overhead and scaling as the number of pairs grows.

7.5 FINAL REMARKS

When a learned discrete code never changes, an outside observer can learn what its messages mean. Episode-dependent remapping tests whether the link between transmitted identifiers and task states can be made to shift from episode to episode while Bob still decodes correctly. The preliminary results justify further work. Verification, and attackers stronger than the two used here, have to come before anyone relies on the approach, and the project is best read as a starting point for measuring privacy mechanisms in learned multi-agent communication.

References

[1] Abadi, M., & Andersen, D. G. (2016). Learning to protect communications with adversarial neural cryptography. *arXiv preprint arXiv:1610.06918*. https://arxiv.org/abs/1610.06918

[2] Foerster, J. N., Assael, Y. M., de Freitas, N., & Whiteson, S. (2016). Learning to communicate with deep multi-agent reinforcement learning. *Advances in Neural Information Processing Systems, 29*.

[3] Havrylov, S., & Titov, I. (2017). Emergence of language with multi-agent games: Learning to communicate with sequences of symbols. *Advances in Neural Information Processing Systems, 30*.

[4] Lazaridou, A., Peysakhovich, A., & Baroni, M. (2017). Multi-agent cooperation and the emergence of (natural) language. *International Conference on Learning Representations*.

[5] Liu, Y., et al. (2023). Adaptive discrete communication bottlenecks with dynamic vector quantization for heterogeneous representational coarseness. *Proceedings of the AAAI*
Conference on Artificial Intelligence, 37(7). https://doi.org/10.1609/aaai.v37i7.26061

[6] Sukhbaatar, S., Szlam, A., & Fergus, R. (2016). Learning multiagent communication with backpropagation. *Advances in Neural Information Processing Systems, 29*.

[7] van den Oord, A., Vinyals, O., & Kavukcuoglu, K. (2017). Neural discrete representation learning. *Advances in Neural Information Processing Systems, 30*.

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*.

[9] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems, 30*.

[10] Das, A., Gervet, T., Romoff, J., Batra, D., Parikh, D., Rabbat, M., & Pineau, J. (2019). TarMAC: Targeted multi-agent communication. *Proceedings of the 36th International Conference on Machine Learning*.

[11] Hernandez-Leal, P., Kartal, B., & Taylor, M. E. (2019). A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems, 33*, 750–797.

[12] Lowe, R., Foerster, J., Boureau, Y.-L., Pineau, J., & Dauphin, Y. (2019). On the pitfalls of measuring emergent communication. *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*.

[13] Kottur, S., Moura, J. M. F., Lee, S., & Batra, D. (2017). Natural language does not emerge 'naturally' in multi-agent dialog. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*.

[14] Mordatch, I., & Abbeel, P. (2018). Emergence of grounded compositional language in multi-agent populations. *Proceedings of the AAAI Conference on Artificial Intelligence, 32*(1).

[15] Mnih, V., et al. (2016). Asynchronous methods for deep reinforcement learning. *Proceedings of the 33rd International Conference on Machine Learning*.

[16] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. *International Conference on Learning Representations*.

[17] Diffie, W., & Hellman, M. E. (1976). New directions in cryptography. *IEEE Transactions on Information Theory, 22*(6), 644–654. https://doi.org/10.1109/TIT.1976.1055638

[18] Shannon, C. E. (1949). Communication theory of secrecy systems. *Bell System Technical Journal, 28*(4), 656–715. https://doi.org/10.1002/j.1538-7305.1949.tb00928.x

APPENDIX A DATASET GENERATION AND PROJECT CONFIGURATION

This appendix presents the configuration and dataset preparation stages of the secure robot communication project. The system consists of Alice, the sender; Bob, the receiver; and Eve, the eavesdropper. The communication task uses feature vectors to represent task states and discrete symbols to transmit information between the communicating agents. The dataset generation stage creates task-state samples belonging to multiple classes. These samples are used to train the communication model and evaluate the ability of Bob and Eve to infer the

intended task state. The project configuration defines the number of classes, input feature dimensions, codebook size, random seeds, and computational device.

A.1 PROJECT CONFIGURATION

The following code illustrates how the experiment configuration and reproducibility settings can be defined.

Listing A.1: Project Configuration and Reproducibility Settings

```
from pathlib import Path import random import numpy as np
import torch

SEED = 42
NUM_CLASSES = 10
INPUT_DIM = 16
CODEBOOK_SIZE = 16
BATCH_SIZE = 128

DEVICE = torch.device(
   "cuda" if torch.cuda.is_available() else "cpu"
)

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
   torch.cuda.manual_seed_all(SEED)

PROJECT_ROOT = Path(".")
OUTPUT_DIR = PROJECT_ROOT / "outputs"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

print("Device:", DEVICE) print("Output
directory:", OUTPUT_DIR.resolve())
```

A.2 DATASET STRUCTURE

Each sample consists of a feature vector and an associated task-state label. The following example demonstrates a prototype-based dataset in which samples are generated by adding noise to class-specific feature vectors. It illustrates the general structure of the data pipeline; the exact data-generation logic should be replaced with the implementation used in the final experiment.

Listing A.2: Representative Communication Dataset from torch.utils.data import Dataset

```
class CommunicationDataset(Dataset):

   def __init__(
      self,
      num_samples=5000,
num_classes=10,
input_dim=16,
noise_std=0.10,        seed=42
   ):
      rng = np.random.default_rng(seed)

      self.prototypes                =                rng.normal(
size=(num_classes, input_dim)
      ).astype(np.float32)

      self.labels = rng.integers(
         0, num_classes, size=num_samples
      )

      noise = rng.normal(
         0,            noise_std,
size=(num_samples, input_dim)
).astype(np.float32)

      self.features                    =                    (
self.prototypes[self.labels] + noise
      ).astype(np.float32)

   def __len__(self):
      return len(self.labels)

   def __getitem__(self, index):
      x = torch.tensor(self.features[index])
y = torch.tensor(
self.labels[index],
dtype=torch.long
      )
```

return x, y

The dataset provides the input representations and task-state labels required for model training. The same class definitions and data-generation parameters must be used consistently when preparing training, validation, and test datasets.

APPENDIX B ALICE–BOB COMMUNICATION ARCHITECTURE

This appendix describes the neural communication architecture used to represent the sender and receiver. Alice encodes the input task state into a discrete communication symbol, while Bob decodes the received representation to predict the corresponding task state. A finite codebook allows the agents to communicate using a limited vocabulary of discrete symbols. The neural architecture is composed of fully connected layers and nonlinear activation functions. The following code is a representative implementation based on the architecture discussed in the project.

B.1 ALICE ENCODER

Alice receives the task-state feature vector and produces scores corresponding to the available communication symbols. The symbol with the highest score is selected as the transmitted discrete symbol. **Listing B.1: Alice Encoder** import torch import torch.nn as nn import torch.nn.functional as F

```
class AliceEncoder(nn.Module):

    def __init__(
        self,
        input_dim=16,
hidden_dim=64,
codebook_size=16
    ):
        super().__init__()

        self.network = nn.Sequential(
nn.Linear(input_dim, hidden_dim),
nn.ReLU(),              nn.Linear(hidden_dim,
codebook_size)
        )

    def forward(self, state_features):

        logits = self.network(state_features)

        symbol_id = torch.argmax(
            logits,
dim=-1
        )

        return symbol_id, logits
```

B.2 BOB DECODER

Bob receives a representation of the transmitted symbol and predicts the original task-state class. The output layer produces one score for each possible task class. **Listing B.2: Bob Decoder** class BobDecoder(nn.Module):

```
    def __init__(
        self,
        codebook_size=16,
hidden_dim=64,
num_classes=10
    ):
        super().__init__()
```

```
        self.network = nn.Sequential(
nn.Linear(codebook_size, hidden_dim),
nn.ReLU(),              nn.Linear(hidden_dim,
num_classes)
        )

    def forward(self, symbol_one_hot):

        logits = self.network(symbol_one_hot)

        return logits
```

B.3 DISCRETE SYMBOL REPRESENTATION

A discrete symbol identifier can be converted into a one-hot vector before being passed to the decoder. Each vector contains a single active position corresponding to the selected symbol.

Listing B.3: One-Hot Symbol Encoding def

```
one_hot_symbols(
    symbol_ids,
codebook_size=16
):
    return F.one_hot(
symbol_ids.long(),
num_classes=codebook_size
    ).float()
```

B.4 CODEBOOK UTILIZATION

Monitoring symbol usage helps determine whether the learned communication protocol uses the available codebook effectively. The following function counts the number of times each symbol appears and calculates the number of distinct symbols used. **Listing B.4: Symbol Usage Analysis** def symbol_usage(symbol_ids, codebook_size=16):

```
    counts = torch.bincount(        symbol_ids.reshape(-
1).long(),        minlength=codebook_size
    )

    used_symbols = int(
        (counts > 0).sum().item()
    )
```

```
    return {
        "counts": counts.cpu().tolist(),
        "used_symbols": used_symbols,
        "codebook_size": codebook_size
    }
```

The utilization analysis complements receiver accuracy by indicating how effectively the learned protocol uses its discrete vocabulary. A high receiver accuracy alone does not establish that every available symbol is being used.

APPENDIX C EPISODE-DEPENDENT SYMBOL REMAPPING

This appendix describes the symbol-remapping mechanism used to vary the transmitted symbol identifiers across communication episodes. In the rotating configuration, Alice applies an episode-dependent permutation to the original symbol identifier. Bob uses the corresponding inverse permutation to recover the original identifier before decoding. Let (s) represent the original symbol and let (\pi_e) denote the permutation associated with episode (e). The transmitted symbol is defined as:

[c=\pi_e(s)]

The receiver recovers the original symbol using:

[s=\pi_e^{-1}(c)]

This remapping changes the observable symbol identifiers while preserving the underlying communication meaning for a receiver that possesses the correct permutation.

C.1 EPISODE SEED GENERATION

The following example derives a deterministic seed from a shared seed and the episode identifier. Deterministic seed generation makes experiments reproducible. It should not be interpreted as a cryptographically secure key-management mechanism. **Listing C.1: Episode-Dependent Seed Generation** import hashlib

def episode_seed(shared_seed, episode_id):

```
    material = (
f"{shared_seed}:{episode_id}"
    ).encode("utf-8")

    digest = hashlib.sha256(material).digest()

    return int.from_bytes(
digest[:8],
byteorder="big"
    )
```

C.2 PERMUTATION AND INVERSE PERMUTATION

A permutation rearranges the available symbol identifiers. The inverse permutation records the original identifier associated with each remapped identifier.

Listing C.2: Permutation Generation

```
import random

def make_permutation(codebook_size, seed):

    permutation = list(
range(codebook_size)
    )

    rng = random.Random(seed)
rng.shuffle(permutation)

    inverse = [0] * codebook_size

    for original_symbol, transmitted_symbol in enumerate(
permutation
    ):
        inverse[transmitted_symbol] = original_symbol

    return permutation, inverse
```

C.3 SYMBOL TRANSMISSION AND RECOVERY

Alice uses the permutation to transform the original symbol identifier. Bob applies the inverse permutation to recover it.

Listing C.3: Sender and Receiver Remapping def
transmit_symbol(symbol_id, permutation):

```
    return permutation[symbol_id]


def recover_symbol(
transmitted_id,
inverse_permutation
):

    return inverse_permutation[transmitted_id]
```

C.4 ROUND-TRIP VALIDATION

The following test checks whether every symbol can be recovered after applying the permutation and its inverse.

Listing C.4: Permutation Correctness Test

```
def test_round_trip(
codebook_size=16,
seed=123
):

    permutation, inverse = make_permutation(
codebook_size,         seed
    )

    for symbol in range(codebook_size):

        transmitted = transmit_symbol(
symbol,              permutation
        )

        recovered = recover_symbol(
transmitted,             inverse
        )

        assert recovered == symbol

    return True


assert test_round_trip()
```

print("Permutation round-trip test passed.")
The round-trip test verifies the correctness of the remapping implementation. It does not independently establish security against an eavesdropper. Security evaluation requires testing against the specified attacker models and comparing their performance across communication configurations.

APPENDIX D TRAINING, EVALUATION AND EAVESDROPPER ANALYSIS

This appendix presents representative routines for training and evaluating the communication models. The evaluation process measures the receiver's ability to recover the intended task state and the eavesdropper's ability to infer the task state from intercepted symbols.
The results must be computed from the final trained models and a held-out test dataset. The code excerpts below illustrate the evaluation logic and must be adapted to the actual model interfaces in the project.

D.1 CLASSIFIER TRAINING

The following example illustrates a standard supervised training loop for a receiver model. It assumes that Alice has already been trained or is being held fixed while Bob is trained. The variable alice must refer to an initialized Alice model, and the data loader must provide compatible feature vectors and labels.
Listing D.1: Representative Receiver Training Loop

```
import torch import torch.nn as nn

def train_receiver(
alice,    bob,
dataloader,
optimizer,
device,
epochs=20
):

   criterion = nn.CrossEntropyLoss()
```

```
    alice.to(device)
bob.to(device)

    alice.eval()

    for epoch in range(epochs):

        bob.train()

        total_loss          =          0.0
total_items = 0

        for state_features, labels in dataloader:

            state_features      =      state_features.to(device)
labels = labels.to(device)

            optimizer.zero_grad()

            with torch.no_grad():
                symbol_ids, _ = alice(state_features)

                symbol_vectors          =          one_hot_symbols(
symbol_ids
                ).to(device)

            logits = bob(symbol_vectors)

            loss = criterion(logits, labels)

            loss.backward()
optimizer.step()

            total_loss          +=          (
loss.item() * labels.size(0)
            )

            total_items += labels.size(0)

        mean_loss      =      total_loss      /      max(
total_items, 1
```

```
    )
print(
        f"Epoch {epoch + 1}/{epochs}: "
f"Loss = {mean_loss:.4f}"
    )
```

This example is intended to illustrate receiver-only training with Alice held fixed. It is not a replacement for the project's complete training procedure if Alice and Bob are jointly trained or use a different discrete-symbol learning mechanism.

D.2 CLASSIFICATION ACCURACY

Accuracy is calculated as the fraction of correctly predicted labels among all evaluated samples.

Listing D.2: Accuracy Evaluation

```
@torch.no_grad() def
evaluate_classifier(
model, dataloader,
device ):

  model.eval()

  correct = 0
total = 0

  for inputs, labels in dataloader:

    inputs = inputs.to(device)
labels = labels.to(device)

    logits = model(inputs)

    predictions = torch.argmax(
      logits,
dim=1
    )

    correct += (
predictions == labels
    ).sum().item()

    total += labels.numel()
```

```
    accuracy = correct / max(total, 1)

    return accuracy
```

The receiver and eavesdropper must be evaluated using the appropriate inputs for their respective roles. Bob receives the representation available to the legitimate receiver, whereas Eve receives only the intercepted information permitted by the threat model.

D.3 RANDOM-GUESSING BASELINE

For a task with ten equally likely classes, uniform random guessing has an expected accuracy of 10%. If the test classes are imbalanced, this baseline should be supplemented with the majority-class prediction accuracy.

Listing D.3: Reference Baseline Calculations

```
import
numpy as np

def uniform_random_baseline(num_classes=10):

    return 1.0 / num_classes


def majority_class_baseline(test_labels):

    labels = np.asarray(test_labels)

    if    labels.size    ==    0:
return float("nan")

    _, counts = np.unique(
labels,
return_counts=True
    )

    return counts.max() / counts.sum()
```

D.4 EXPERIMENTAL RESULT REPORTING

The final evaluation should report receiver accuracy, analytical eavesdropper accuracy, neural eavesdropper accuracy, symbol utilization, dataset split sizes, and relevant training settings for each communication configuration.

All comparisons should use consistent test data and clearly documented experimental conditions. The random-guessing reference must reflect the actual number of classes and class distribution. Any claimed reduction in eavesdropper accuracy should be supported by

the executed evaluation outputs rather than assumed from the design of the remapping mechanism.

The results must also distinguish between deterministic symbol remapping and cryptographic security. Episode-dependent permutation can make direct symbol-to-state associations less stable for an observer without the current mapping, but it does not by itself guarantee resistance to every possible attack.

APPENDIX E EXPERIMENTAL FRAMEWORK, SECURITY ANALYSIS AND LIMITATIONS

E.1 OVERVIEW OF THE EXPERIMENTAL FRAMEWORK

The experimental framework investigates whether a learned discrete communication protocol can enable communication between agents while limiting the inferential capabilities of an external observer. The framework revolves around three agents: Alice, Bob, and Eve. Alice encodes a representation of the current task state into a discrete symbol from a finite codebook. The encoded symbol is passed through a communication channel and decoded by Bob, who reconstructs an estimate of the initial task state. Eve observes the transmitted symbols and attempts to infer the task state.

The framework explores variations of the described setup and evaluates the trade-off between the communication performance (receiver accuracy) and the ability to prevent such performance (eavesdropper accuracy). The learned protocol is not fully specified by a manually designed dictionary. Instead, Alice transforms the input state representation into an encoder representation that selects a symbol from the codebook. Bob then learns to associate the received symbols with task-state classes.

The following section describes the learned discrete communication in more detail. The experimental comparison utilizes several communication configurations, which are discussed next. Various factors affect the ability of an observer to infer information from discrete symbols, including the properties of the codebook and the relationship between the encoded information and the chosen symbols.

E.2 LEARNED DISCRETE COMMUNICATION

Many communication systems are based on conventional encodings of information into symbols. Learned discrete communication enables neural agents to develop task-specific associations between their internal representations and a finite set of discrete symbols. In the proposed protocol, Alice encodes the input state representation into an encoder representation that selects one of the available symbols from the codebook. Bob learns to decode the received symbol into a decoder representation and perform the task of inferring the initial task state.

The learned communication protocol can be viewed as a mapping function that transforms the encoder representations into discrete symbols. The same encoder representation can be mapped to different symbols depending on the specific communication configuration. Multiple encoder representations can correspond to the same symbol, particularly if the

learned parameters fail to fully utilize the available codebook capacity. Thus, effective communication critically depends on the ability of Alice to encode the relevant information into the encoder representation and Bob's ability to decode the symbol and reconstruct the task state.

Codebook utilization is an important aspect of discrete communication. While the codebook contains a fixed set of available symbols, the utilization of the codebook by the encoder and decoder typically follows a particular distribution. This distribution should be analyzed when assessing the efficacy of a particular communication protocol. Codebook utilization should not be conflated with receiver accuracy. High receiver accuracy indicates that Bob successfully decoded the information and performed the task. However, analyzing the utilization of individual symbols provides additional insights into the characteristics of the learned codebook representation.

E.3 COMMUNICATION CONFIGURATIONS

The experimental comparison utilizes several communication configurations that differ in how the transmitted symbols are mapped to the codebook identifiers. The details of each communication configuration are as follows.

E.3.1 Plain Communication

With plain communication, Alice directly transmits the learned symbol identifier learned. Bob subsequently receives the transmitted representation and performs decoding and task inference with the learned decoder. Plain communication serves as the baseline for evaluating the effect of various symbol remappings on communication performance and eavesdropper accuracy.

An observer with access to the transmitted symbol identifier can attempt to learn associations between the observed symbol identifiers and the corresponding task classes by analyzing the frequency of occurrence of particular symbols. Such analysis depends on the task and the distribution of the labels. Thus, plain communication can be utilized to assess the baseline inference performance against which the performance of other configurations can be compared.

E.3.2 Fixed Permutation

Fixed-permutation communication utilizes an additional permutation of the codebook identifiers. The permutation changes the observed identifiers associated with each symbol. With a fixed permutation, an observer with access to the communication channel can learn the association between the permuted symbol identifiers and the original labels. Thus, the value of such a configuration primarily depends on the particular choice of permutation. The fixed permutation approach primarily allows for a change in the representation of the

communication protocol rather than limiting inferential capabilities directly. A permutation changes the associations between the observed symbol identifiers and the original labels.
An observer with sufficient training data can learn these associations and utilize them to infer the labels from the observed identifiers. The choice of a particular permutation primarily affects the rate at which such associations can be learned.

E.3.3 Key-Dependent Static Remapping

The key-dependent configuration utilizes a permutation that depends on a particular key that was used to initialize the random number generator. The details of such a permutation are consistent throughout the lifetime of a single evaluation episode. Thus, a key-dependent static remapping configuration changes the associations between the observed identifiers and the actual labels in a similar manner to the fixed permutation configuration.
The key-dependent configuration serves as a way to distinguish between a permutation that consistently changes the associations between the observed identifiers and the original labels and a permutation that changes these associations across different episodes. With a consistent permutation, the associations between the observed identifiers and the original labels remain consistent throughout all training examples.
As a result, the inference accuracy of the eavesdropper should be similar to the case of utilizing a fixed permutation. However, the key-dependent configuration provides additional insights into the overall effectiveness of a particular remapping approach. The utilization of such an approach provides meaningful security guarantees only if the key is not known to the attacker.

E.3.4 Episode-Dependent Rotating Remapping

Episode-dependent rotating remapping utilizes a different permutation for every episode. The choice of a particular permutation depends on the episode identifier and the deterministic seed generation procedure discussed below. Alice utilizes the chosen permutation to encode the selected symbol identifier before transmitting it to Bob, who utilizes the inverse permutation to decode the received symbol identifier.
The primary goal of episode-dependent rotating remapping is to limit the statistical information that can be inferred about the associations between the observed symbol identifiers and the actual labels. With a single observation, an attacker would be able to infer the label only if they could correctly guess the permutation that was utilized to encode the symbol identifier. A change in permutation across different episodes limits the ability of an attacker to utilize frequency analysis to infer the associations between the labels and the permuted identifiers.
However, a different approach might allow an attacker to infer the associations across episodes. If an observer can correctly identify the episode boundaries and utilize additional information about the distribution of the labels, the utilization of different permutations across episodes provides limited protection. Thus, episode-dependent rotating remapping primarily serves as an approach to reduce the value of direct observations of permuted

identifiers rather than providing strong theoretical guarantees about the security of the communication protocol.

E.4 THREAT MODEL AND ATTACKER CAPABILITIES

The current framework assumes that Eve observes the discrete symbols transmitted through the communication channel. However, she cannot directly access the internal state of either Alice or Bob. The observer attempts to utilize the observed information to infer the corresponding information state that is relevant to the task performed by Alice and Bob.

The eavesdropper can utilize two approaches to perform such inference: analytical and neural. An analytical approach typically involves explicitly modeling the observed associations between the observed symbols and the information states. A neural approach, on the other hand, typically involves training a classifier to directly infer the information states from the observed symbols. Both approaches provide meaningful insights into the security of the communication protocol. However, analytical approaches provide stronger theoretical guarantees in cases where such guarantees are possible.

The threat model does not explicitly consider the possibility of Eve compromising either Alice or Bob directly. Similarly, the threat model does not consider the possibility of the attacker intercepting or modifying the shared secret key, observing the memory contents of either agent, or utilizing denial-of-service attacks to disrupt the communication between Alice and Bob. Thus, the security guarantees provided by the current framework only consider the observations of discrete symbols directly transmitted through the communication channel.

The deterministic seed-generation procedure utilized in the current framework primarily serves to ensure reproducibility of results. However, such an approach does not provide strong security guarantees for the communication protocol. In particular, a single shared secret key should not be utilized for long-lasting or high-security applications. A more comprehensive threat model would explicitly consider the secrecy of the shared key and additional aspects of the system design.

E.5 PERFORMANCE EVALUATION CRITERIA

The evaluation of the performance of the communication protocols primarily revolves around two criteria: receiver accuracy and eavesdropper accuracy.

Receiver accuracy is a critical metric that characterizes the performance of the communication protocol. Ideally, the receiver accuracy should approach 100% for any evaluated configuration. The utilization of a particular remapping approach without significant receiver accuracy improvements typically indicates that the protocol fails to provide meaningful benefits for communication.

On the other hand, eavesdropper accuracy serves as a metric that characterizes the security of the communication protocol. For a given threat model, lower eavesdropper accuracy typically indicates better security. Thus, the analysis should consider the baseline accuracy that can be achieved by a randomly guessing attacker. Assuming that there are ten possible

classes with equal probability, an attacker that always guesses the same class would be correct in approximately 10% of cases, on average. A similar baseline accuracy should be evaluated for imbalanced class distributions, with the majority class serving as the baseline. Other relevant evaluation criteria include symbol utilization and computational complexity. Symbol utilization provides additional insights into the characteristics of the communication protocol. In particular, the evaluation should report how many of the available symbols are actually utilized in the communication process. Computational complexity, on the other hand, primarily affects the practical applicability of a given approach.

Thus, the analysis should explicitly state the utilized training and test data splits, random seeds, and the number of training epochs. Similarly, the accuracy scores should be based on the final evaluation rather than the expected performance of the randomly initialized model. In particular, the evaluation script should utilize the final trained weights rather than the randomly initialized weights when calculating the accuracy scores.

E.6 LIMITATIONS OF THE EXPERIMENTAL DESIGN

The current framework involves several limitations that should be considered when interpreting the results. First, the performance of the communication protocol fundamentally depends on the specifics of the task. In particular, the analysis should consider how the described approach generalizes to more complex tasks and larger state spaces. Similarly, a higher observation noise or a more challenging task might significantly impact the performance of the communication protocol.

Second, the utilization of a discrete codebook limits the amount of information that can be transmitted by either Alice or Bob. In particular, the codebook might not be fully utilized if the trained encoder fails to adequately utilize all available entries. Thus, the performance of the communication protocol critically depends on the choice of the codebook size. Increasing the codebook size typically increases the complexity of the task without providing significant benefits for the communication process itself.

Third, the analysis primarily considers two particular attacker models that utilize either analytical or neural approaches. In reality, an attacker might utilize a combination of approaches, additional information about the task, or different assumptions about the problem. Thus, the analysis should explicitly state the threat model utilized in the experiments rather than assuming that a particular approach covers all possible attacks.

Fourth, the rotating remapping approach fundamentally depends on the correct utilization of the permutations. In particular, the security guarantees only hold if Bob correctly receives the information about the current permutation. Furthermore, the analysis does not consider whether the utilized secret-sharing approach is secure. Thus, the analysis should explicitly state that rotating remapping is only secure if the underlying secret sharing approach is secure.

Fifth, deterministic remapping should not be confused with secure encryption. In particular, changing the associations between the observed identifiers and the transmitted identifiers does not provide any guarantees about the security of the communication process itself.

Thus, the described approach is primarily focused on limiting the direct inference of information from the observed identifiers rather than providing end-to-end security guarantees.

Finally, all analysis fundamentally depends on the ability to correctly reproduce the results. Thus, the training and evaluation procedures need to be carefully designed to ensure consistency across different runs. In particular, the analysis should utilize the same random seeds and data splits when comparing different configurations. Furthermore, the results need to be validated across different random seeds to ensure that the observed trends are not caused by chance.

E.7 REPRODUCIBILITY AND FUTURE EXTENSIONS

Reproducibility plays a critical role in scientific research. Thus, the current experimental framework should include the necessary information, scripts, and checkpoints to enable complete reproducibility of the results. In particular, the analysis should explicitly state the details of the model architecture, codebook size, random seeds, data generation procedures, permutation strategies, and evaluation metrics. Furthermore, the saved model checkpoints and prediction files should be associated with the particular set of configuration parameters. The training and evaluation scripts should be separated, where applicable. The training script defines the communication protocol, whereas the evaluation script determines how the receiver accuracy and the eavesdropper accuracy should be calculated. Thus, changes to the evaluation script should not affect the training process.

The current analysis primarily focuses on the characteristics of the discussed communication protocol. Further research should be devoted to evaluating how such a protocol generalizes to larger task spaces and more complex environments. Future work could also focus on designing a communication protocol that effectively utilizes multiple time steps rather than a single observation.

The evaluation of such a protocol would benefit from additional attacker models that explicitly incorporate additional information about the task. The utilization of independent test and calibration data sets would also improve the reproducibility and validity of the results. A future extension of the current analysis could explicitly compare the described protocol against existing security benchmarks to evaluate whether the described approach provides reasonable security guarantees under a particular attacker model.